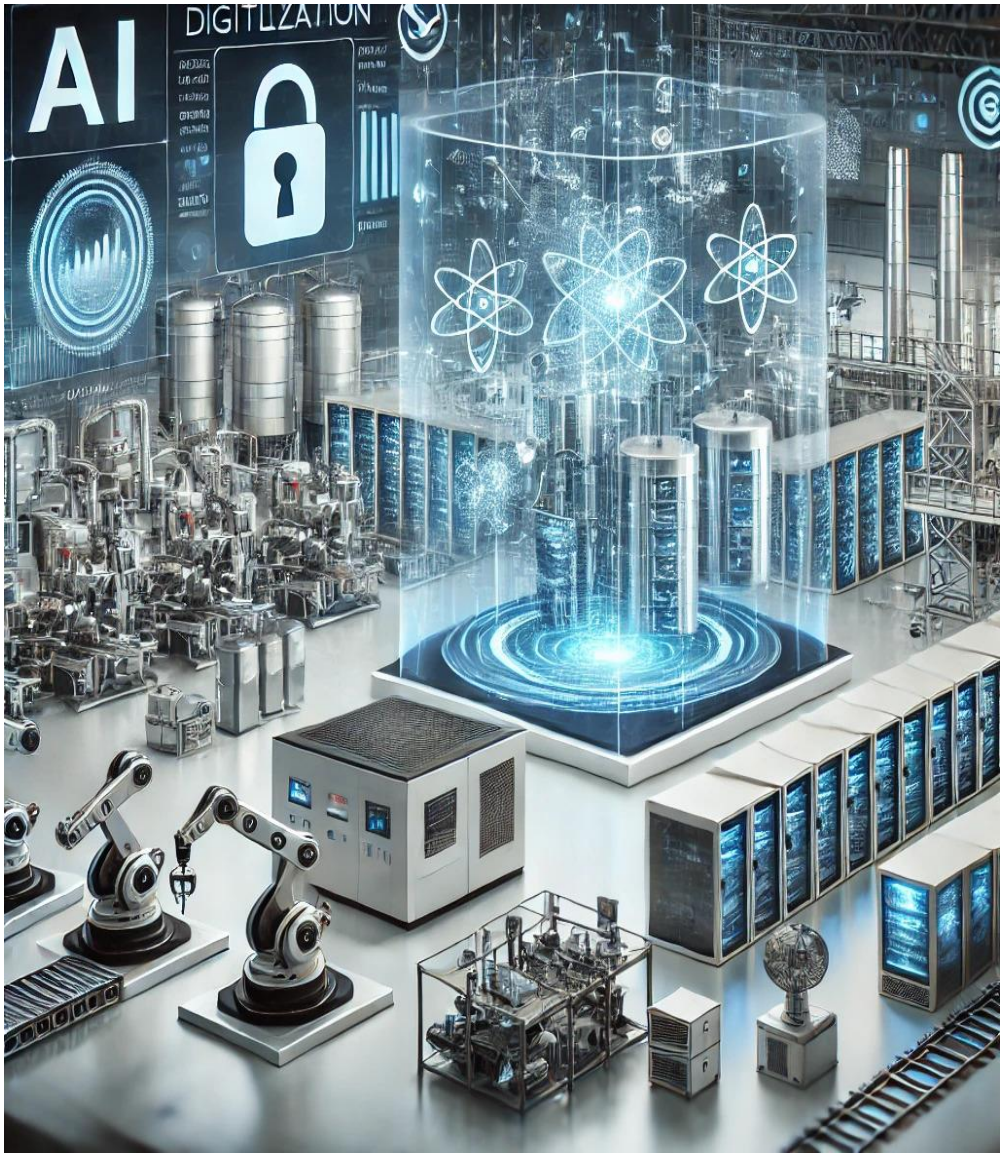


Hochschule für
Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Embedded Systems Engineering Verifikation

Prof. Dr.-Ing. Sebastian Schlesinger
Professor of Business Computer Science
Cybersecurity and Embedded Systems Engineering

Überblick

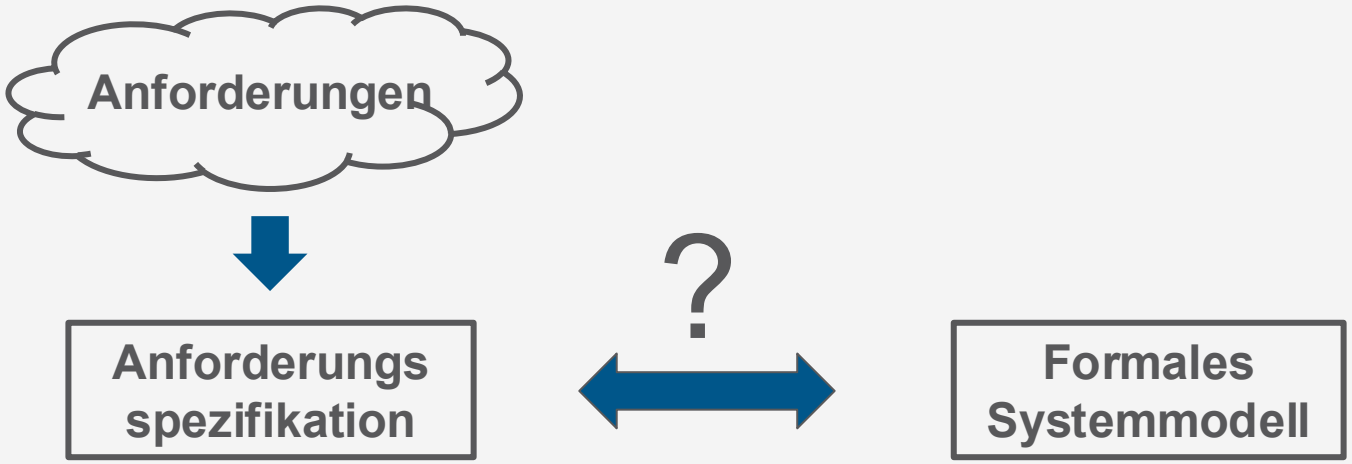
- Einführung: Formale Verifikation
- Formales Systemmodell
  - Kripke-Struktur
  - Von C Code zu Kripke-Strukturen
  - Von synchronen Schaltkreisen zu Kripke-Strukturen
- Anforderungsspezifikation
  - Linear Temporal Logic (LTL)
  - Computation Tree Logic (CTL)
- Model Checking

Überblick

- Einführung: Formale Verifikation
- Formales Systemmodell
  - Kripke-Struktur
  - Von C Code zu Kripke-Strukturen
  - Von synchronen Schaltkreisen zu Kripke-Strukturen
- Anforderungsspezifikation
  - Linear Temporal Logic (LTL)
  - Computation Tree Logic (CTL)
- Model Checking

Formale Verifikation

- Ziel: zeigen, dass ein Modell des Systems gegebene Eigenschaften (Anforderungen) erfüllt



- Schritte:
  1. ein formales Modell des Systems konstruieren
  2. erwünschtes Verhalten / Eigenschaften formalisieren
  3. verifizieren, dass das System die Anforderungen erfüllt

Theorembeweisen

(u.U. unendliches)
Modell

M

P

Logische Eigenschaften

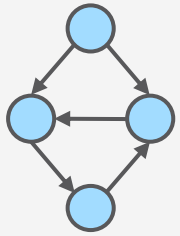
Theorem
beweiser
(interaktiv)

Eigenschaften nicht erfüllt
($M \wedge P \Rightarrow$ False)
oder kein Beweis gefunden

Eigenschaften erfüllt
(Beweis für $M \Rightarrow P$)

Model Checking

endliches Zustandsmodell



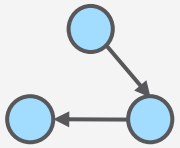
G c

G (a ⇒ X b)

Temporallogische Eigenschaften

Model Checker (automatisch)

Gegenbeispiel



Eigenschaften erfüllt

Werkzeuge zur formalen Verifikation

- Theorembeweiser
  - System und Anforderungen beschrieben in formaler Logik
  - Verifikation durch die Anwendung von Inferenzregeln (Ableitungsregeln)
  - Interaktive Beweisführung: Logiken höherer Ordnung, Verifikation unendlicher Systeme mit z.B. Induktion
  - Automatische Beweisführung: Logiken erster Ordnung, SAT/SMT solving

- Model Checker
  - Systemmodell muss auf endlichen Automat abgebildet werden
  - Temporale Logik zur Beschreibung von Eigenschaften
  - Verifikation vollautomatisch

Theorembeweisen

(u.U. unendliches)
Modell

M

P

Logische Eigenschaften

Theorem
beweiser
(interaktiv)

Eigenschaften nicht erfüllt
($M \wedge P \Rightarrow$ False)
oder kein Beweis gefunden

Eigenschaften erfüllt
(Beweis für $M \Rightarrow P$)

Formale Logik

- Struktur: Logik = Syntax + Semantik + Kalkül
- Syntax: Was ist eine Formel?
  (welche syntaktischen Elemente dürfen verwendet werden?)
- Semantik: Was bedeutet eine Formel?
  (ist eine Formel wahr oder falsch?)
- Kalkül
  - Syntax und Semantik
  - Axiome
  - Inferenzregeln

Aussagenlogik Syntax

Definition (Syntax der Aussagenlogik)

Atomare Formeln bezeichnen wir mit $A, B, \ldots$

Die Menge der Formeln wird induktiv definiert:

1. $\perp$ und T sind Formeln.
2. Alle atomaren Aussagen sind Formeln.
3. Für alle Formeln F und G sind auch $(F \wedge G)$ und $(F \vee G)$ Formeln.
4. Für jede Formel F ist auch $\neg F$ eine Formel.

Aussagenlogik Semantik

Definition (Semantik der Aussagenlogik)

- $\mathcal{A}(\bot)$ = False, $\mathcal{A}(\mathrm{T})$ = True
- $\mathcal{A}(F)$ = $\mathcal{A}(F)$, wenn F eine atomare Aussage ist
- $\mathcal{A}(\neg F)$ = True, wenn $\mathcal{A}(F)$ = False
  = False, sonst
- $\mathcal{A}(F \wedge G)$ = True, wenn $\mathcal{A}(F)$ = True und $\mathcal{A}(G)$ = True
  = False, sonst
- $\mathcal{A}(F \vee G)$ = True, wenn $\mathcal{A}(F)$ = True oder $\mathcal{A}(G)$ = True
  = False, sonst

Beweistheorie

Ziel: syntaktische Beschreibung von Schlussfolgerungen als Ableitung in einem Regelsystem

Beispiel:
Wenn der ÖPNV streikt und mein Fahrrad kaputt ist,
komme ich zu spät. Der ÖPNV streikt. Ich bin pünktlich.
Dann wissen wir:
Mein Fahrrad ist nicht kaputt.

Warum wissen wir, dass das Fahrrad nicht kaputt ist?

Beweistheorie

Ziel: syntaktische Beschreibung von Schlussfolgerungen als Ableitung in einem Regelsystem

Beispiel:
Wenn der ÖPNV streikt und mein Fahrrad kaputt ist,
komme ich zu spät. Der ÖPNV streikt. Ich bin pünktlich.
Dann wissen wir:
Mein Fahrrad ist nicht kaputt.

$$\frac{(F_1 \wedge F_2 \rightarrow F_3), F_1, \neg F_3}{\neg F_2}$$

F_1 Der ÖPNV streikt.
F_2 Mein Fahrrad ist kaputt.
F_3 Ich komme zu spät.

Warum wissen wir, dass das Fahrrad nicht kaputt ist?

Kalkül

- Sprache mit formaler Syntax und Semantik (z.B. Aussagenlogik, Prädikatenlogik)
- Axiome
- Inferenzregeln:

$$\frac{A \quad A \rightarrow B}{B}$$

Prämisse

Schlussfolgerung

- Beweis: systematische Ableitung einer Aussage in einem Kalkül (Aufbau eines Beweisbaumes)
- Vorwärtsschließen: Anwenden einer Regel auf die Beweisannahmen
- Rückwärtsschließen: Anwenden einer Regel auf das Beweisziel
- Beispielkalkül: Kalkül des natürlichen Schließens

Kalkül des natürlichen Schließens ($\wedge, \vee, \rightarrow$)

	Introduction	Elimination
$\wedge$	$\dfrac{\phi \qquad \psi}{\phi \wedge \psi}$	$\dfrac{\phi \wedge \psi}{\phi}$ $\qquad$ $\dfrac{\phi \wedge \psi}{\psi}$
$\vee$	$\dfrac{\phi}{\phi \vee \psi}$ $\qquad$ $\dfrac{\psi}{\phi \vee \psi}$	$\dfrac{\phi \vee \psi \qquad \begin{array}{c}[\phi]^1 \\ \vdots \\ \chi\end{array} \qquad \begin{array}{c}[\psi]^2 \\ \vdots \\ \chi\end{array}}{\chi}$
$\rightarrow$	$\dfrac{\begin{array}{c}[\phi]^1 \\ \vdots \\ \psi\end{array}}{\phi \rightarrow \psi}$	$\dfrac{\phi \qquad \phi \rightarrow \psi}{\psi}$

Kalkül des natürlichen Schließens ($\neg$, $\bot$,$\top$)

	Introduction	Elimination	
$\neg$	$\dfrac{\begin{array}{c}[\phi]^1\\ \vdots\\ \bot\end{array}}{\neg\phi}$	$\dfrac{\phi \quad \neg\phi}{\bot}$	$\dfrac{\begin{array}{c}[\neg\phi]^1\\ \vdots\\ \bot\end{array}}{\phi}$
$\neg\neg$		$\dfrac{\neg\neg\phi}{\phi}$	
$\bot$		$\dfrac{\bot}{\phi}$	
$\top$	$\dfrac{}{\top}$		

Beweistheorie

Wenn der ÖPNV streikt und mein Fahrrad kaputt ist, komme ich zu spät. Der ÖPNV streikt. Ich bin pünktlich.

F_1 Der ÖPNV streikt.

F_2 Mein Fahrrad ist kaputt.

F_3 Ich komme zu spät.

$$(F_1 \wedge F_2 \rightarrow F_3) \quad F_1 \quad \neg F_3$$

Können wir das durch Anwendung der Inferenzregeln ableiten?

Mein Fahrrad ist nicht kaputt.

$$\overline{\neg F_2}$$

Beweistheorie

Wenn der ÖPNV streikt und mein Fahrrad kaputt ist, komme ich zu spät. Der ÖPNV streikt. Ich bin pünktlich.

F_1 Der ÖPNV streikt.
F_2 Mein Fahrrad ist kaputt.
F_3 Ich komme zu spät.

$$
\begin{array}{lr}
(F_1 \land F_2 \to F_3) \quad F_1 \quad \neg F_3 & \\
(F_1 \land F_2 \to F_3) \quad F_1 \quad \neg F_3 \quad \boxed{F_2} & \neg\,\mathrm{i} \\
(F_1 \land F_2 \to F_3) \quad \boxed{F_1 \land F_2} \quad \neg F_3 & \land\,\mathrm{i} \\
(F_3) \quad \neg F_3 & \to \mathrm{e} \\
\bot & \neg\,\mathrm{e} \\
\hline
\neg F_2 &
\end{array}
$$

Mein Fahrrad ist nicht kaputt.

Eigenschaften eines Kalküls

- **Korrektheit** eines Kalküls:
  Es können nur gültige Aussagen abgeleitet werden.
- **Vollständigkeit** eines Kalküls
  Es können alle gültigen Aussagen abgeleitet werden.

Verbreitete Kalküle für Eingebettete Systeme

Hoare-Kalkül

- Idee: Effekt von einzelnen Programmschritten als Inferenzregeln
- Vor- u. Nachbedingungen eines Programms + Invarianten ⇨ Programmverifikation

Prozesskalküle (wie z.B. CSP und der pi-Kalkül)

- Idee: Entwicklung von Prozessen als Inferenzregeln
- Korrektheit als Verfeinerungsbeziehungen

Differential Dynamic Logic (dL) und der dL Verification Calculus

- Idee: hybride Systeme als diskrete Zuweisungen + Differentialgleichungen
- Inferenzregeln beschreiben die möglichen Entwicklungen von diskreten und kontinuierlichen Variablen
- zur Verifikation von diskreten Komponenten in physikalischer Umgebung

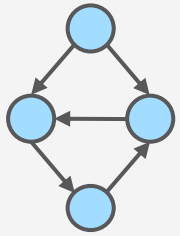
Logiken erster Ordnung

- Wie beschreibt man ein Schachspiel (32 Figuren, 64 Felder, ~100 Züge) ?
- ~100 000 000 000 000 000 000 000 000 000 000 000 000 Seiten als Zustand-zu-Zustand Transitions-Matrix (z.B. Markov Modelle, Automaten)
- ~100 000 Seiten in Aussagenlogik (z.B. Schaltkreise, graphische Modelle)
  WhiteKingOnC4@Move12
- 1 Seite in einer Logik erster Ordnung
  On(color,piece,x,y,t)

[Quelle: Stuart Russell]

Model Checking

endliches Zustandsmodell



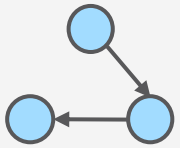
G c

G (a ⇒ X b)

Temporallogische Eigenschaften

Model Checker (automatisch)

Gegenbeispiel



Eigenschaften erfüllt

Gegenbeispiel

- ein Gegenbeispiel ist ein *trace* des Systems, der eine in der Anforderungsspezifikation geforderte Eigenschaft verletzt
- Gegenbeispiele können oft wesentlich schneller erzeugt werden, als erfüllte Eigenschaften gezeigt werden können
- die Verletzung einer Eigenschaft kann auf einen Fehler im Modell oder auf einen Fehler in der Anforderungsspezifikation hinweisen

Model Checking

- beim Model Checking wird geprüft, ob ein Modell $\mathcal{M}$ eine Eigenschaft φ erfüllt:

$$\mathcal{M} \models \varphi$$

- das Modell $\mathcal{M}$ ist eine Kripke Struktur (*labeled state transition graph*)
- die Eigenschaften werden in temporaler Logik ausgedrückt
- die Kripke Struktur wird meistens nicht explizit verwendet
- stattdessen verwenden wir Spezifikationssprachen, die Kripke Strukturen beschreiben (z.B. Automaten, Statecharts, Timed Automata)

Überblick

- Einführung: Formale Verifikation
- Formales Systemmodell
  - Kripke-Struktur
  - Von C Code zu Kripke-Strukturen
  - Von synchronen Schaltkreisen zu Kripke-Strukturen
- Anforderungsspezifikation
  - Linear Temporal Logic (LTL)
  - Computation Tree Logic (CTL)
- Model Checking

Kripke-Strukturen

- Eine Kripke-Struktur ist ein Tupel (S,T,S_0,L) mit
  - S einer Menge von Zuständen
  - $T \subseteq S \times S$ einer Menge von Transitionen
  - $S_0 \subseteq S$ eine Menger von Initialzuständen
  - $L : S \rightarrow 2^{AP}$ einer Markierungsfunktion
    (L(s) : Menge der atomaren Präpositionen, die in s erfüllt sind)

- Damit Model Checking möglich ist, muss S endlich sein.

- Pfad einer Kripke-Struktur:
  $\pi = s_0 s_1 s_2 \ldots$ ist ein Pfad von (S,T,S_0,L) falls $\forall i. (s_i, s_{i+1}) \in T$

Von C Code zur Kripke-Struktur

```
1  int x = 0;
2
3  void thread() {
4    int v = x;
5    x = v + 1;
6  }
7
8  void main() {
9    fork(thread);
10   int u = x;
11   x = u + 1;
12   join(thread);
13   assert(x == 2);
14 }
```

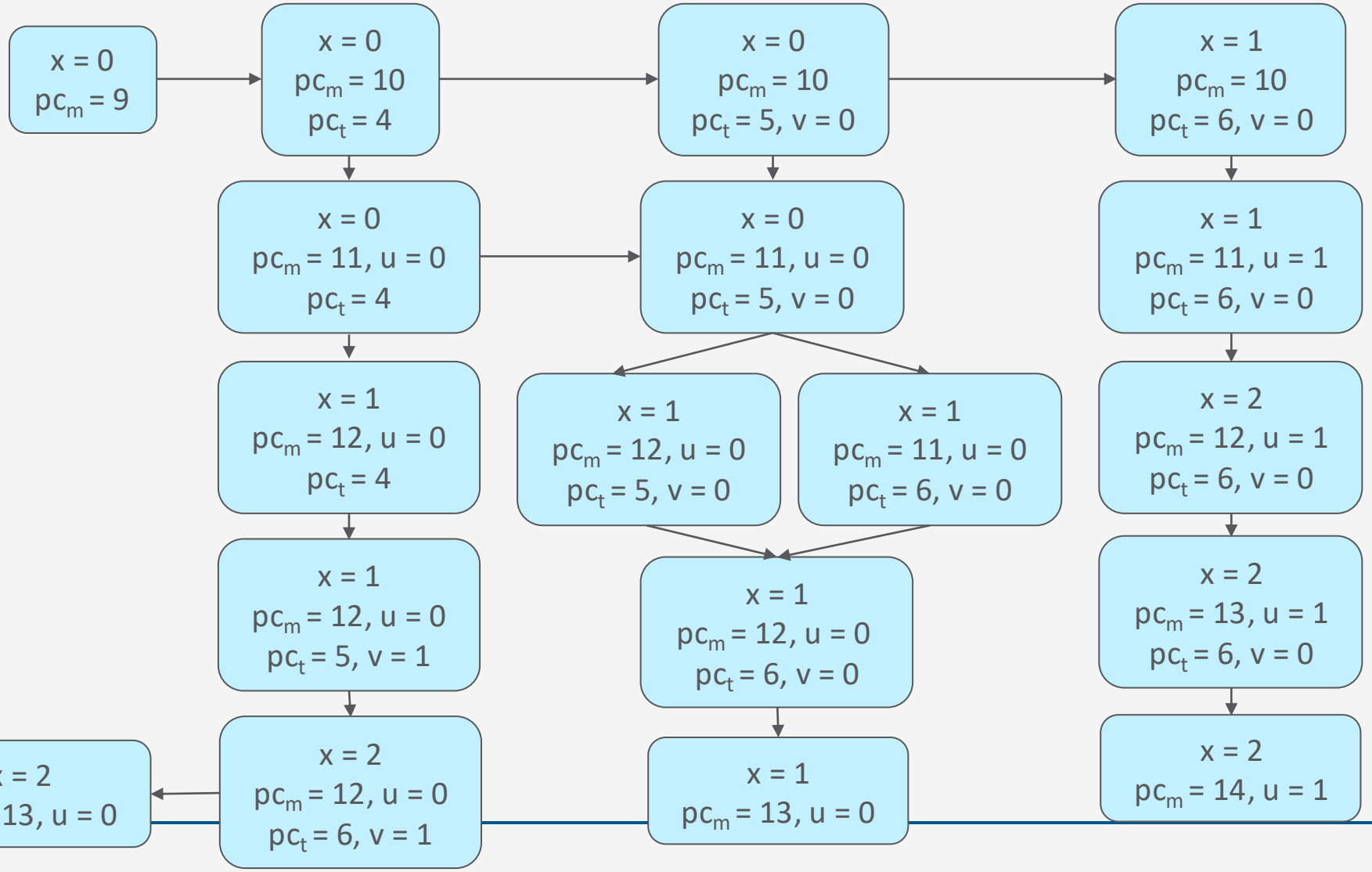
In welchen **Zuständen** kann dieses Programm sein?

Die Zustände eines C-Programms können als Tupel (pc, v) beschrieben werden, wobei pc der *program counter* ist und v ein Vektor von Variablenbelegungen.

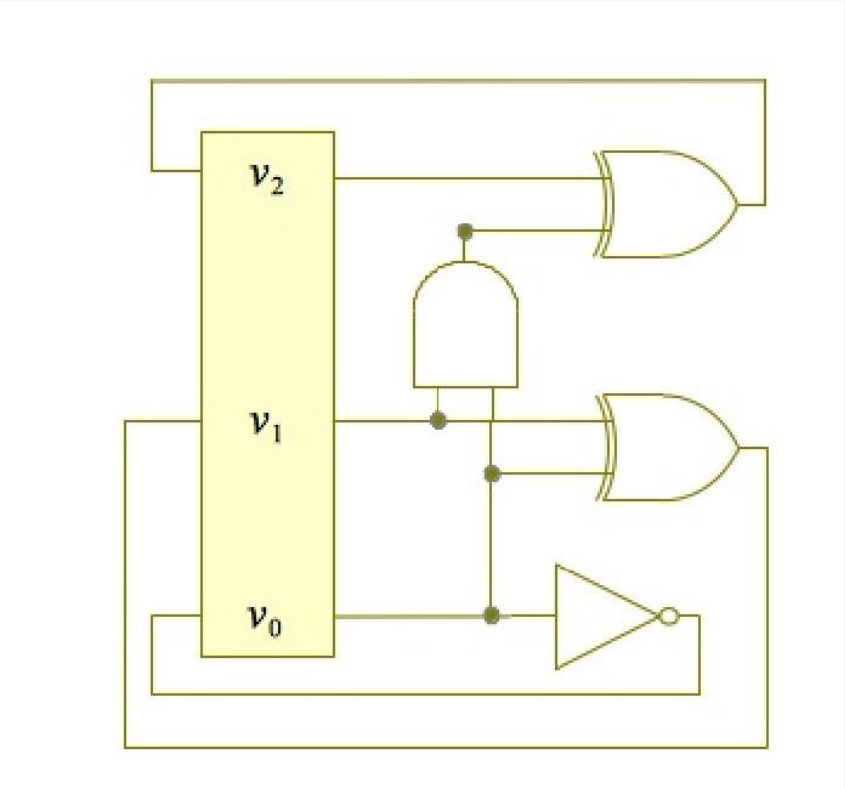
Berechnen Sie die möglichen Zustände des Programms!
Der Initialzustand ist $(pc = 9, x = 0)$.

Von C Code zur Kripke-Struktur

```
1  int x = 0;
2
3  void thread() {
4    int v = x;
5    x = v + 1;
6  }
7
8  void main() {
9    fork(thread);
10   int u = x;
11   x = u + 1;
12   join(thread);
13   assert(x == 2);
14 }
```

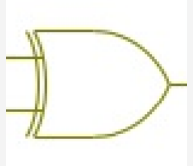


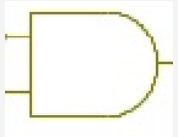
Vom synchronen Schaltkreis zur Kripke-Struktur

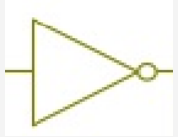


In welchen **Zuständen** kann dieser Schaltkreis sein?

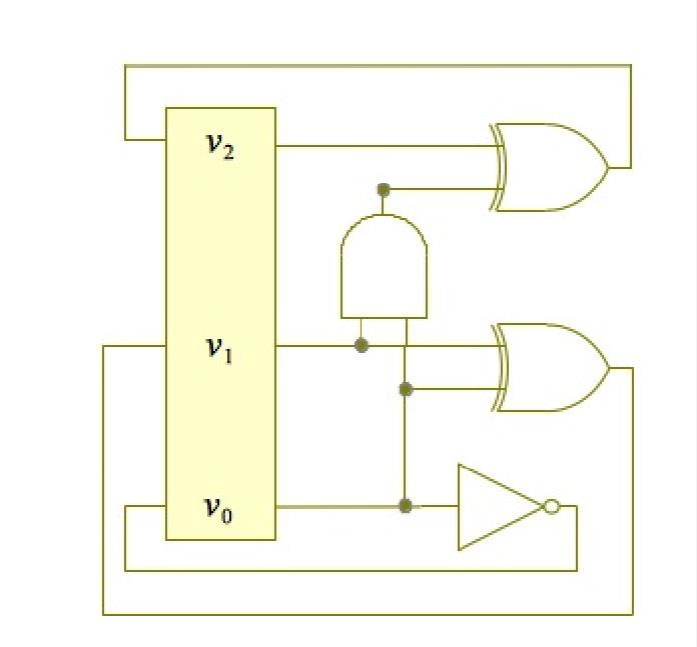
Die Zustände eines (synchronen) Schaltkreises können als Vektor (v) der Registerinhalte beschrieben werden.


XOR


AND


NOT

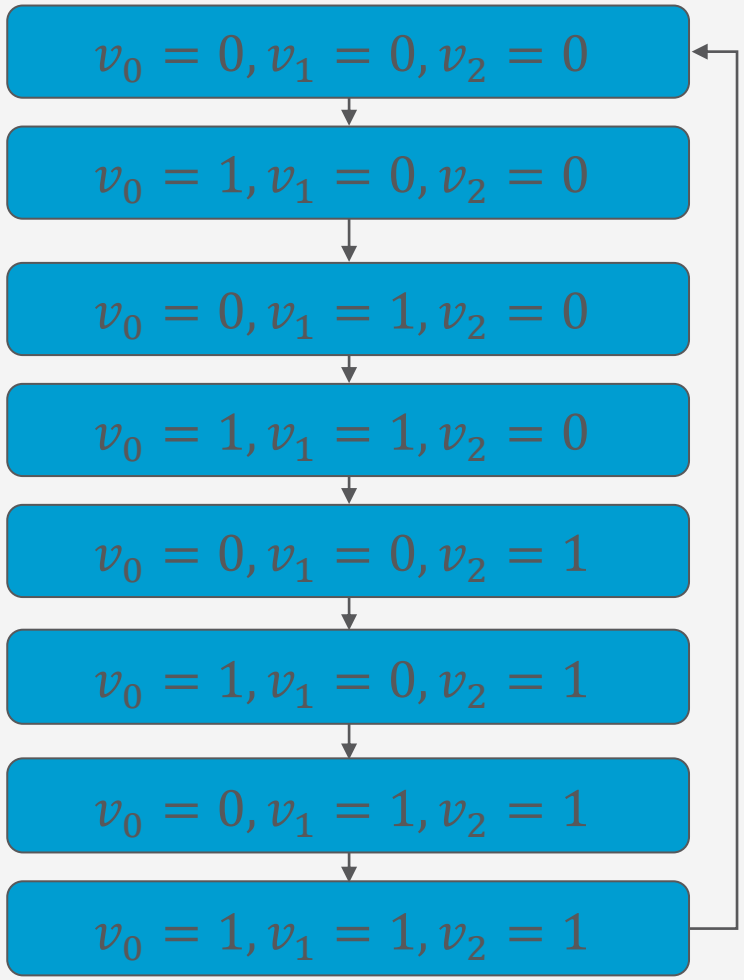
Vom synchronen Schaltkreis zur Kripke-Struktur



$$v_2' = (v_0 \wedge v_1) \oplus v_2$$

$$v_1' = v_0 \oplus v_1$$

$$v_0' = \neg v_0$$



Überblick

- Einführung: Formale Verifikation
- Formales Systemmodell
  - Kripke-Struktur
  - Von C Code zu Kripke-Strukturen
  - Von synchronen Schaltkreisen zu Kripke-Strukturen
- Anforderungsspezifikation
  - Linear Temporal Logic (LTL)
  - Computation Tree Logic (CTL)
- Model Checking

Eigenschaften (*properties*)

- Eigenschaften werden als zeitliche / temporale Eigenschaften über Pfaden formuliert
- Beispiele
  - das System antwortet immer innerhalb von 100 ms
  - es befindet sich immer nur ein Prozess im kritischen Abschnitt
  - ein Fehlerzustand wird niemals erreicht
  - von jedem Zustand ist ein bestimmter Zustand erreichbar
  - immer wenn eine Anfrage erfolgt, erfolgt auch irgendwann eine Antwort

Klassifikation von Eigenschaften

- Sicherheit: *something bad will never happen*
  - es befindet sich immer nur ein Prozess im kritischen Abschnitt
  - ein Fehlerzustand wird niemals erreicht
- Lebendigkeit: *something good will eventually happen*
  - eine Anforderung wird irgendwann erfüllt
  - von jedem Zustand ist ein bestimmter Zustand erreichbar
  - immer wenn eine Anfrage erfolgt, erfolgt auch irgendwann eine Antwort
- Fairness: *something is true infinitely often*
  - ein Prozess gibt die Kontrolle unendlich oft ab, so dass auch andere Prozesse abgearbeitet werden können

Temporale Logik

Temporal logics have proven to be useful for specifying concurrent systems, because they can describe the ordering of events in time without introducing time explicitly. [Clarke, Grumberg and Peled 1999]

- Temporale Logik trifft Aussagen wie “auf allen Pfaden gilt …” oder “auf mindestens einem Pfad gilt …”
- Temporale Logik beschreibt Änderungen von Werten über die Zeit „auf einem Pfad gilt immer …“ oder „auf einem Pfad gilt irgendwann“
- Zusammengesetzt z.B. „auf allen Pfaden gilt immer …“, „auf mindestens einem Pfad gilt irgendwann …“

LTL vs. CTL

- zwei Arten von Zeitbegriffen:
  - lineare Zeit:
    - lineare (total geordnete) Folge von Zeitpunkten

      ⇨ Linear-time Temporal Logic (LTL)
  - verzweigte Zeit:
    - verschiedene Folgezustände aus einem Zustand möglich
    - die Menge der möglichen Pfade bildet einen Berechnungsbaum (*computation tree*)

      ⇨ Computation Tree Logic (CTL)

Linear-time Temporal Logic (Syntax)

- induktive Definition von LTL-Formeln:

$\phi ::= \bot \mid \top \mid p \mid (\neg\phi) \mid (\phi\wedge\phi) \mid (\phi\vee\phi) \mid (\phi\Rightarrow\phi) \mid X\,\phi \mid F\,\phi \mid G\,\phi \mid \phi_1\,U\,\phi_2 \mid \phi_1\,W\,\phi_2$

- $\bot$ und $\top$ stehen für wahr und falsch
- p steht für eine atomare Formel (Prädikat)
- die aussagenlogischen Verknüpfungen sind die üblichen
- temporallogische Verknüpfungen:

  $X\,\phi$ "ϕ gilt im nächsten Zustand" (neXt)

  $F\,\phi$ "ϕ gilt in einem zukünftigen Zustand" (Future, auch $\Diamond$ oder <>)

  $G\,\phi$ "ϕ gilt in allen zukünftigen Zuständen" (Globally, auch $\Box$ oder [ ])

  $\phi_1\,U\,\phi_2$ "ϕ_1 gilt in allen Zuständen bis ϕ_2 gilt" (Until)

  $\phi_1\,W\,\phi_2$ "ϕ_1 gilt in allen Zuständen bis ϕ_2 gilt oder ϕ_1 gilt immer" (Weak until)

LTL Formeln

- Welche der LTL Formeln X *p*, F *p* und G *p* sind hier jeweils erfüllt?

S	0	1	2
p	F	T	F

S	0	1	2
p	F	F	T

S	0	1	2
p	T	T	F

S	0	1	2	
p	T	T	T	…

- Welche der LTL-Formeln *p* U *q* und *p* W *q* sind hier jeweils erfüllt?

S	0	1	2
p	T	T	F
q	F	F	T

S	0	1	2
p	T	F	T
q	F	F	T

S	0	1	2
p	T	T	T
q	F	F	F

S	0	1	2
p	F	F	F
q	T	F	F

Semantik von LTL (1)

Let $M = (S, T, S_0, L)$ be a Kripke structure and $\pi = s_0 s_1 s_2 \ldots$ be a path of it.

- $(M, \pi) \models \top$
- $(M, \pi) \models p$ iff $p \in L(s_0)$
- $(M, \pi) \models \neg\phi$ iff $\neg\Big((M, \pi) \models \phi\Big)$
- $(M, \pi) \models (\phi \wedge \psi)$ iff $(M, \pi) \models \phi \wedge (M, \pi) \models \psi$
- etc.

Semantik von LTL (2)

Let $M = (S, T, S_0, L)$ be a Kripke structure and
$\pi = s_0 s_1 s_2 \ldots$ be a path of it.

- $(M, \pi) \models \mathsf{X}\phi$ iff $(M, \pi_1) \models \phi$
- $(M, \pi) \models \mathsf{F}\phi$ iff $\exists i.\ (M, \pi_i) \models \phi$
- $(M, \pi) \models \mathsf{G}\phi$ iff $\forall i.\ (M, \pi_i) \models \phi$
- $(M, \pi) \models \phi \,\mathsf{U}\, \psi$ iff $\exists i.\ (M, \pi_i) \models \psi \wedge \forall j < i.\ (M, \pi_j) \models \phi$
- $(M, \pi) \models \phi \,\mathsf{W}\, \psi$ iff $(M, \pi) \models \phi \,\mathsf{U}\, \psi \vee (M, \pi) \models \mathsf{G}\phi$

We say that $M \models \phi$ iff $\forall \pi = s_0 s_1 s_2 \ldots$ with $s_0 \in S_0$
it holds that $(M, \pi) \models \phi$

Computation Tree Logic (Syntax)

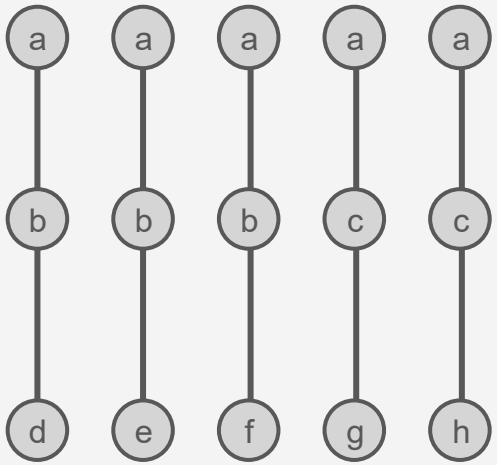
- induktive Definition von CTL-Formeln:

$\phi ::= \bot \mid \top \mid p \mid (\neg\phi) \mid (\phi\wedge\phi) \mid (\phi\vee\phi) \mid (\phi\Rightarrow\phi) \mid$ AX ϕ | EX ϕ | AF ϕ | EF ϕ| AG ϕ | EG ϕ
| A [ϕ U ψ] | E [ϕ U ψ] | A [ϕ W ψ] | E [ϕ W ψ]

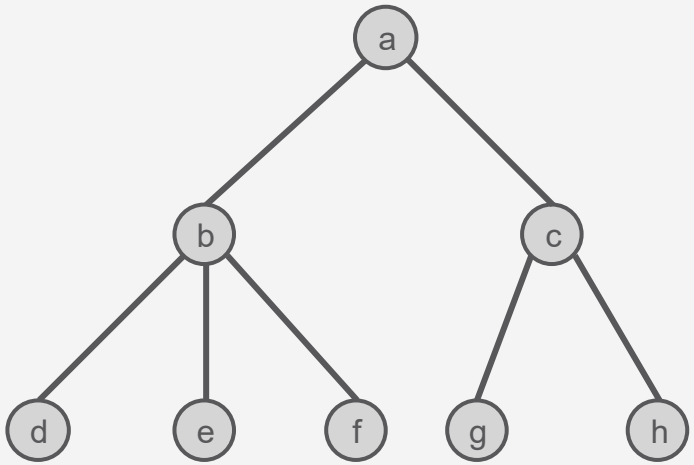
- $\bot$ und $\top$ stehen für wahr und falsch
- p steht für eine atomare Formel (Prädikat)
- die aussagenlogischen Verknüpfungen sind die üblichen
- temporallogische Verknüpfungen wie in LTL
  X ϕ, **F** ϕ, **G** ϕ , ϕ **U** ψ , ϕ **W** ψ
- Pfadquantoren
  - A "entlang aller Pfade" (Always)
  - E "entlang mindestens eines Pfades" (Exists)

LTL vs CTL in Bildern

- Wenn ein System als Kripke-Struktur gegeben ist, wird eine LTL Formel φ als $A\varphi$ interpretiert

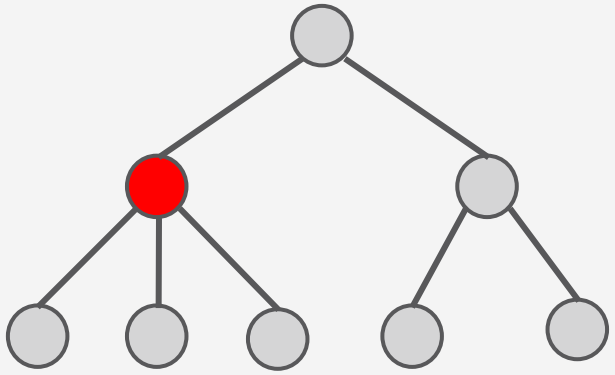


LTL reasons about a set of runs



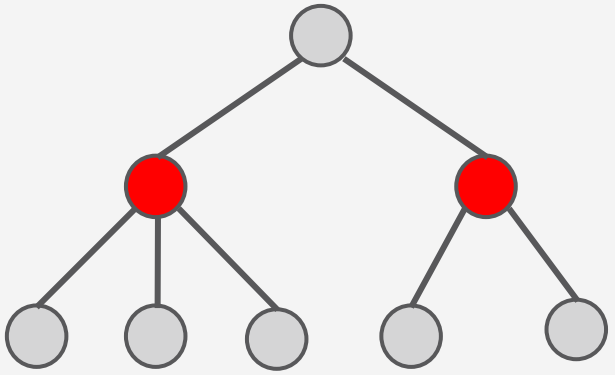
CTL reasons about the computation tree

CTL Operatoren 1/4



EX p

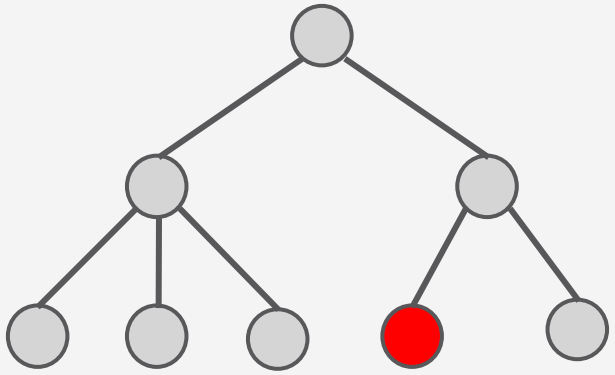
There exist at least one path where p holds in the next state



AX p

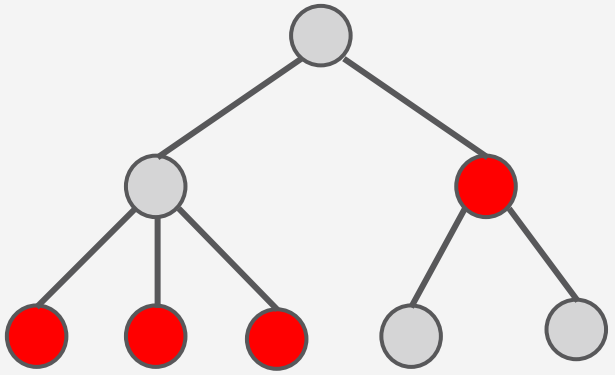
On all paths p holds in the next state

CTL Operatoren 2/4



EF p

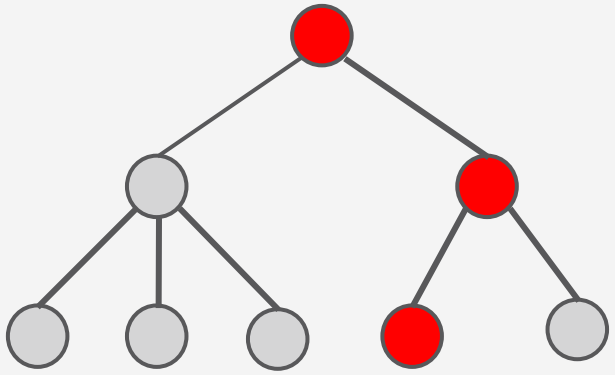
There exists at least one path where p holds eventually in some future state.



AF p

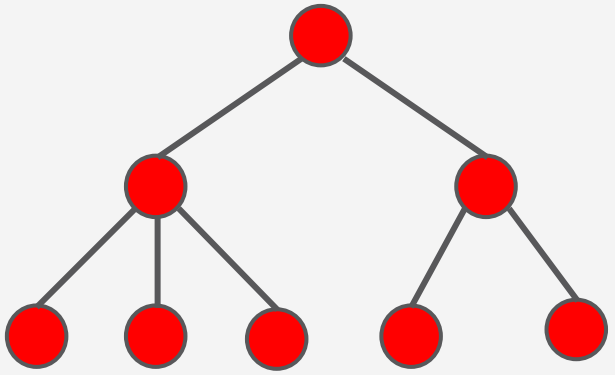
On all paths p holds eventually in some future state.

CTL Operatoren 3/4



EG p

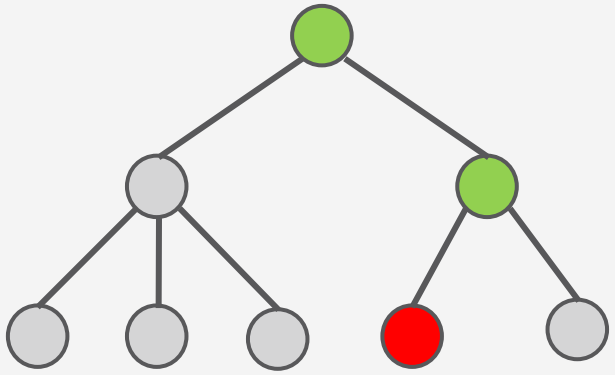
There exists at least one path where p holds globally in all future states.



AG p

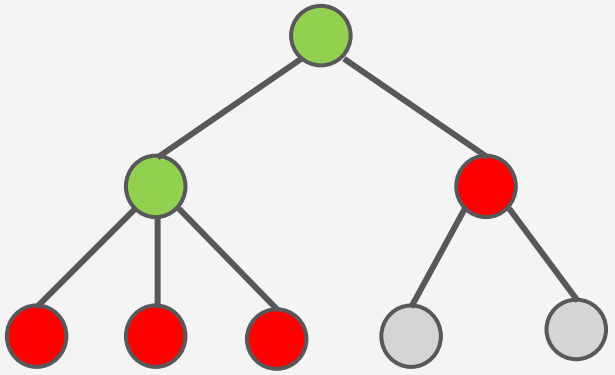
On all paths p holds globally in all future states.

CTL Operatoren 4/4



E [p U q]

There exist at least one path where p holds until q holds.



A [p U q]

On all paths p holds until q holds.

Semantik von CTL (1)

Let $M = (S, T, S_0, L)$ be a Kripke structure and $s \in S$ be a state of it.

- $(M, s) \models \top$
- $(M, s) \models p$ iff $p \in L(s)$
- $(M, s) \models \neg\phi$ iff $\neg\Big((M, s) \models \phi\Big)$
- $(M, s) \models (\phi \wedge \psi)$ iff $(M, s) \models \phi \wedge (M, s) \models \psi$
- etc.

Semantik von CTL (2)

Let $M = (S, T, S_0, L)$ be a Kripke structure and $s_0 \in S$ be a state of it.

- $(M, s_0) \models \mathrm{AX}\phi$ iff for all paths $\pi = s_0 s_1 s_2 \ldots$ in M such that $(M, s_1) \models \phi$
- $(M, s) \models \mathrm{AF}\phi$ iff for all paths $\pi = s_0 s_1 s_2 \ldots$ in M such that $\exists i.\ (M, s_i) \models \phi$
- $(M, s) \models \mathrm{AG}\phi$ iff for all paths $\pi = s_0 s_1 s_2 \ldots$ in M such that $\forall i.\ (M, s_i) \models \phi$
- $(M, s) \models \mathrm{A}[\phi \,\mathrm{U}\, \psi]$ iff for all paths $\pi = s_0 s_1 s_2 \ldots$ in M such that $\exists i.\ (M, s_i) \models \psi \wedge \forall j < i.\ (M, s_j) \models \phi$

Semantik von CTL (3)

Let $M = (S, T, S_0, L)$ be a Kripke structure and $s_0 \in S$ be a state of it.

- $(M, s_0) \models \mathrm{EX}\phi$ iff there exists a path $\pi = s_0 s_1 s_2 \ldots$ in M such that $(M, s_1) \models \phi$
- $(M, s) \models \mathrm{EF}\phi$ iff there exists a path $\pi = s_0 s_1 s_2 \ldots$ in M such that $\exists i.\ (M, s_i) \models \phi$
- $(M, s) \models \mathrm{EG}\phi$ iff there exists a path $\pi = s_0 s_1 s_2 \ldots$ in M such that $\forall i.\ (M, s_i) \models \phi$
- $(M, s) \models \mathrm{E}[\phi \,\mathrm{U}\, \psi]$ iff there exists a path $\pi = s_0 s_1 s_2 \ldots$ in M such that $\exists i.\ (M, s_i) \models \psi \wedge \forall j < i.\ (M, s_j) \models \phi$

We say that $M \models \phi$ iff $\forall s_0 \in S_0.\ (M, s_0) \models \phi$

LTL vs. CTL

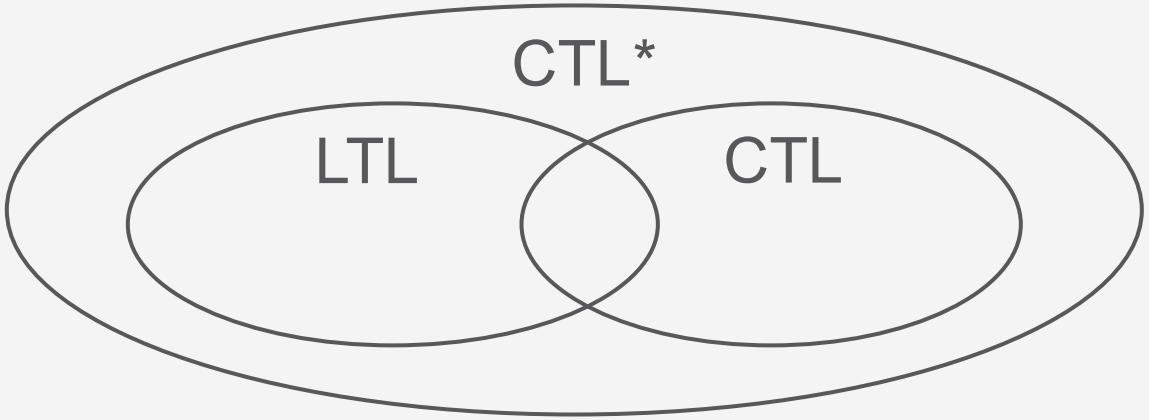
Welche von beiden ist ausdrucksmächtiger?

Finden Sie eine Eigenschaft, die in LTL ausgedrückt werden kann, aber nicht in CTL?

Oder umgekehrt?

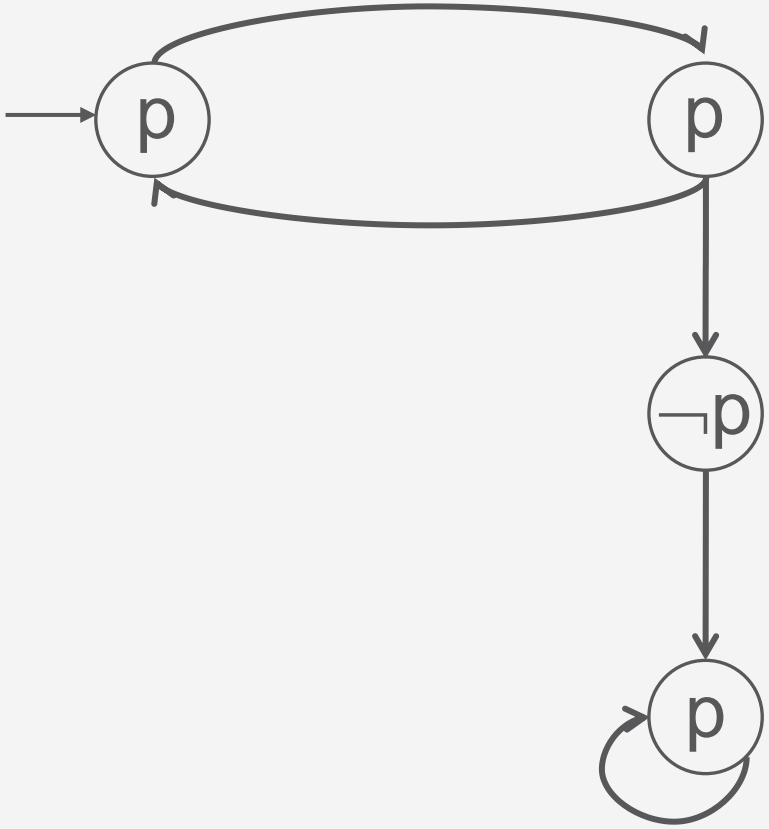
LTL vs. CTL

- CTL Formel, die nicht in LTL ausgedrückt werden kann: EX p
- LTL Formel, die nicht in CTL ausgedrückt werden kann: (A)FG p



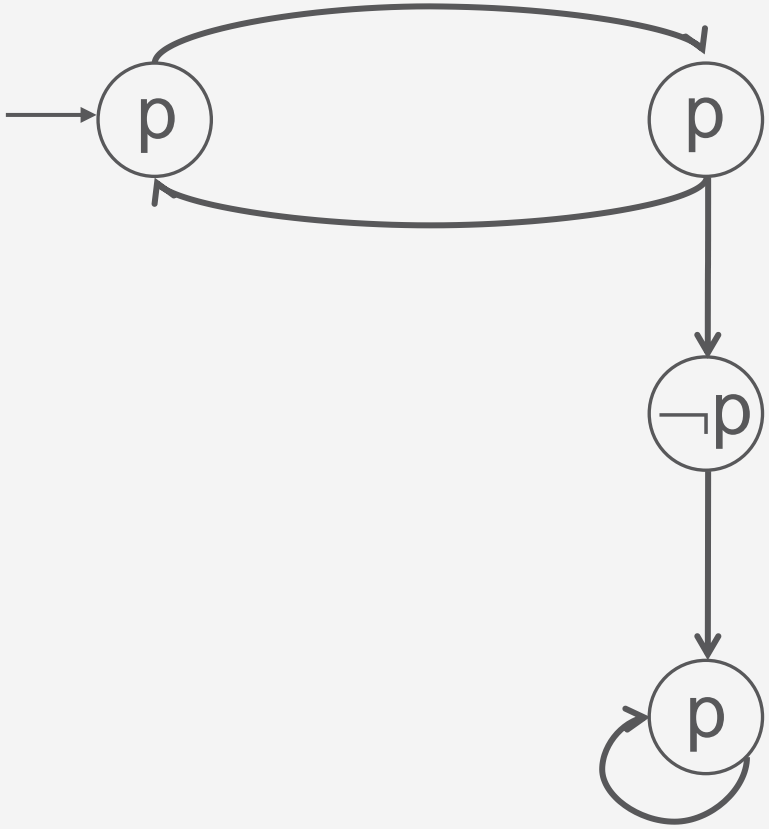
LTL vs. CTL

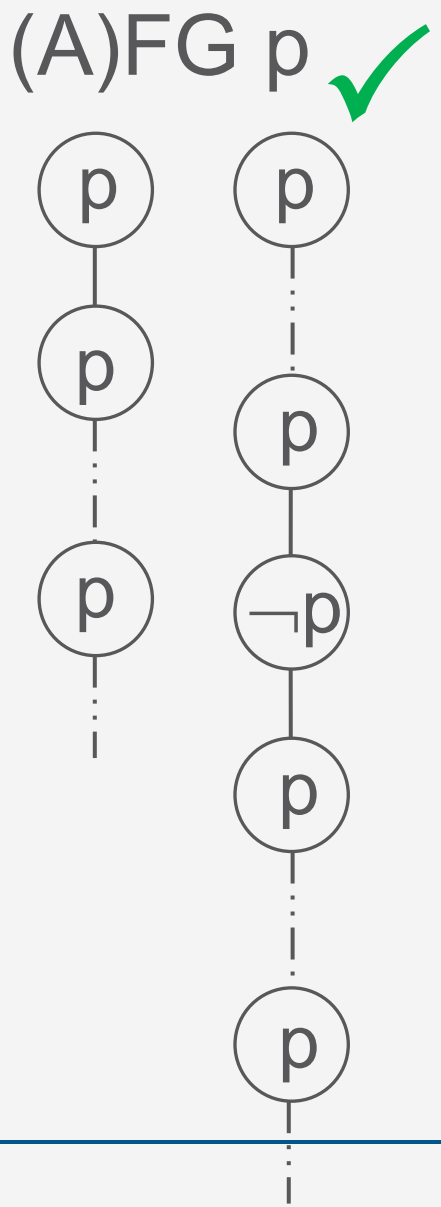
- Ist (A)FG p äquivalent zu AF AG p?

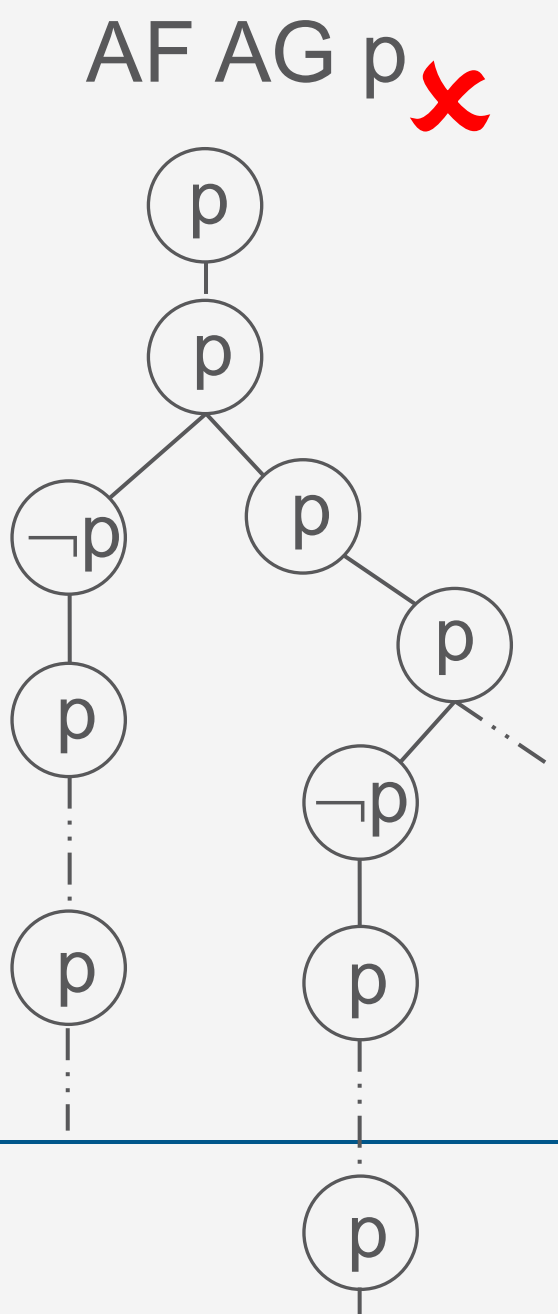


LTL vs. CTL

- Ist (A)FG p äquivalent zu AF AG p?







Timed CTL

- TCTL: Erweiterung von CTL um Zeitbeschränkungen
- Syntax:

$$\phi ::= \bot \mid \top \mid p \mid (\neg\phi) \mid (\phi \wedge \phi) \mid (\phi \vee \phi) \mid (\phi \Rightarrow \phi) \mid A\,[\phi_1\ U\ \ \phi_2] \mid E\,[\phi_1\ U_{\simeq d}\ \phi_2]$$

$\simeq d$: entspricht Uhrenbedingungen in Timed Automata
$AG_{\simeq d}\ \phi == \neg\ E\,[\bot\ U_{\simeq d}\ \neg\phi]$

- mit TCTL können Eigenschaften der Form: “etwas geschieht frühestens nach 10 ms und spätestens nach 200 ms“ ausgedrückt werden
- Beispiel: wenn Alarm ausgelöst wird, geht das System innerhalb von 160 ms in einen Sicherheitszustand über

$$\textit{alarm} \Rightarrow AG_{(\leq 160)}\ \textit{safety_mode}$$

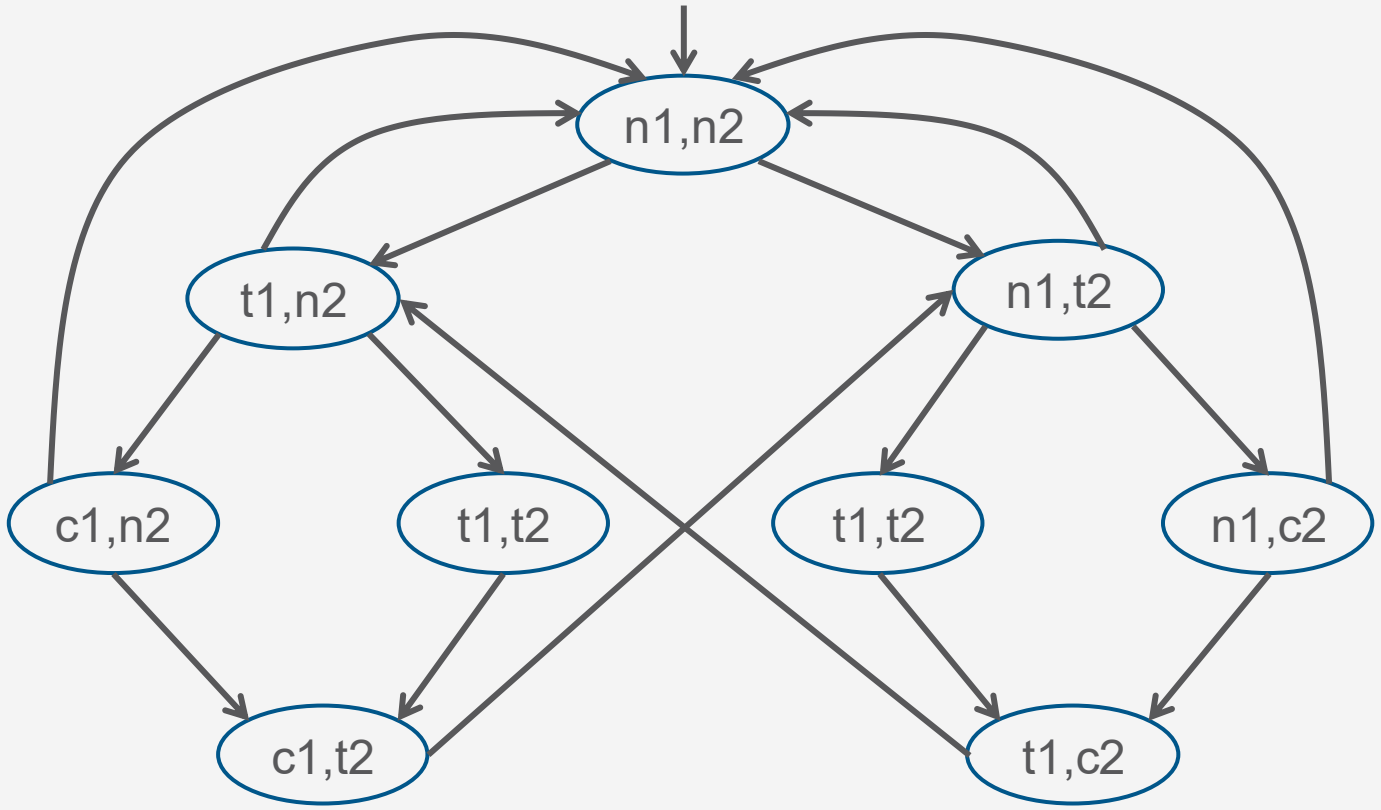
Überblick

- Einführung: Formale Verifikation
- Formales Systemmodell
  - Kripke-Struktur
  - Von C Code zu Kripke-Strukturen
  - Von synchronen Schaltkreisen zu Kripke-Strukturen
- Anforderungsspezifikation
  - Linear Temporal Logic (LTL)
  - Computation Tree Logic (CTL)
- Model Checking

Model Checking: Beispiel

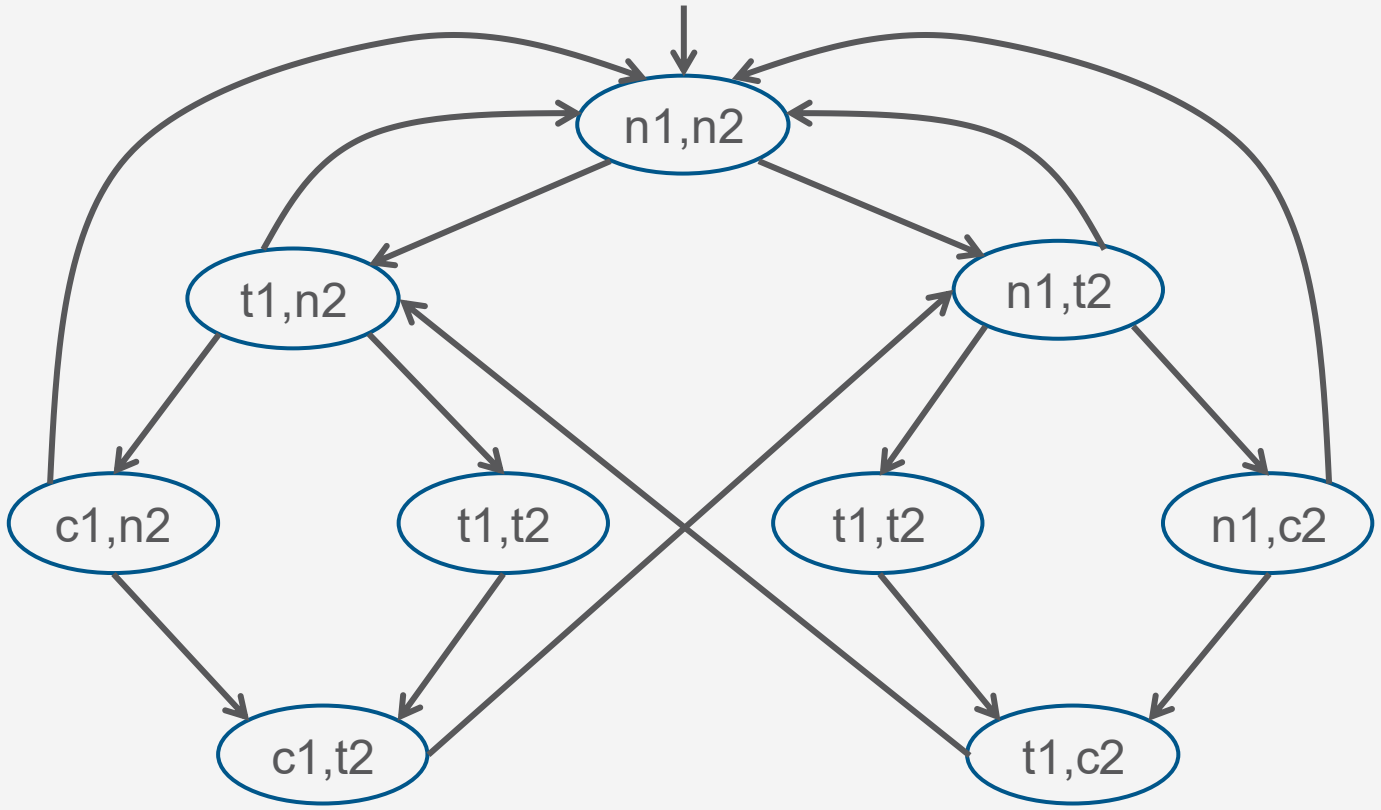
- Beispiel: gegenseitiger Ausschluss
  - zwei Prozesse, i = 1,2
  - Globaler Zustandsraum S1 … S9
  - Atomare Präpositionen:
    - ni: Prozess i ist nicht im kritischen Bereich
    - ti: Prozess i versucht, in den kritischen Bereich zu gelangen
    - ci: Prozess i ist im kritischen Bereich
  - Interleaving Semantik

Model Checking: Beispiel



AG $\neg(c_1 \wedge c_2)$?

Model Checking: Beispiel



Verifikationsprinzip: besuche jeden Zustand und prüfe, ob er die Eigenschaft erfüllt

Große Kripke-Strukturen

- Problem der Zustandsexplosion

- Typische HW-Designs: $2^{1000000}$ Zustände
- Kripke-Struktur ist i.d.R. exponentiell größer als ein Programm
  - Nebenläufige oder parallele Prozesse (kombinatorische Zustandsexplosion)
  - Instantiierung von Variablen
  - Verschachtelte Prozeduren

Lösungsansätze

- Konservativ:
  - Darstellung der Kripke-Struktur durch eine spezielle Datenstruktur
  - ➢ Komprimierung der Informationen
  - ➢ z.B. Symbolisches Model Checking

- Aggressiv:
  - Verwendung von Wissen über die Kripke-Struktur, um von irrelevanten Details zu abstrahieren.
  - ➢ Filterung von Informationen
  - ➢ z.B. Existentielle Abstraktion (Zustände weglassen)

Zusammenfassung

- Theorembeweisen und Model Checking sind verschiedene Paradigmen zur formalen Verifikation
- Theorembeweisen basiert auf einem Kalkül mit Inferenzregeln
- Model Checking faltet den Zustandsraum auf
- Voraussetzung für formale Verifikation ist ein formales Modell des Systems und eine formale Spezifikation der Anforderungen
- eine Kripke-Struktur kann das mögliche Verhalten präzise erfassen (von Software, Hardware, …)
- Temporallogische Formeln erlauben es zu spezifizieren dass etwas “niemals” oder “immer” passiert.
- mit Model Checking können solche Formeln voll-automatisch geprüft werden.
- Problem: Zustandsexplosion

Model Checking vs. Theorembeweisen

Model Checking	Theorembeweisen (HOL)
Eigenschaften eines endlichen Zustandsmodells	deduktive Verifikation
Aussagenlogik	Prädikatenlogik
gründliche (exhaustive) Prüfung des gesamten erreichbaren Zustandsraums	Verwendung syntaktischer Regeln (Induktion, strukturelle Dekomposition)
vollautomatisch	interaktiv, erfordert Expertise
liefert Gegenbeispiele bei Nicht-Erfüllung	Fehler sind schwer zu lokalisieren
Problem der Zustandsexplosion	kann mit unendlichen Wertebereichen umgehen (Abstraktion)

Überblick

- Einführung
- Vorüberlegungen: Eigenschaften von CTL
- Labeling-Algorithmus für CTL
- Model Checking als Fixpunktberechnung
- BDD-based Model Checking
- Zusammenfassung

Motivation

- zur Erinnerung: mit Model Checking können temporallogische Formeln (LTL/CTL) voll-automatisch verifiziert werden
- Model Checking Problem: $\mathcal{M} \models \varphi$
- Idee:
  - "auffalten" einer Kripke-Struktur (*labeled transition system*)
  - Erkundung des Zustandsraums

Aber: wie können wir berechnen, ob eine CTL Formel φ ein gegebenes Modell $\mathcal{M}$ erfüllt?

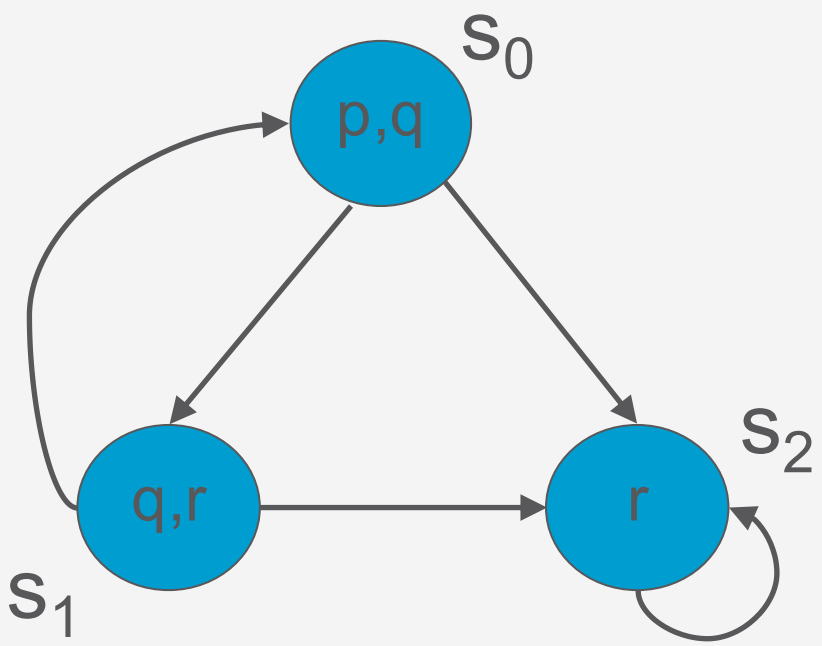
Recap: Kripke Structure

A Kripke structure is a tuple (S, T, S_0, L) where

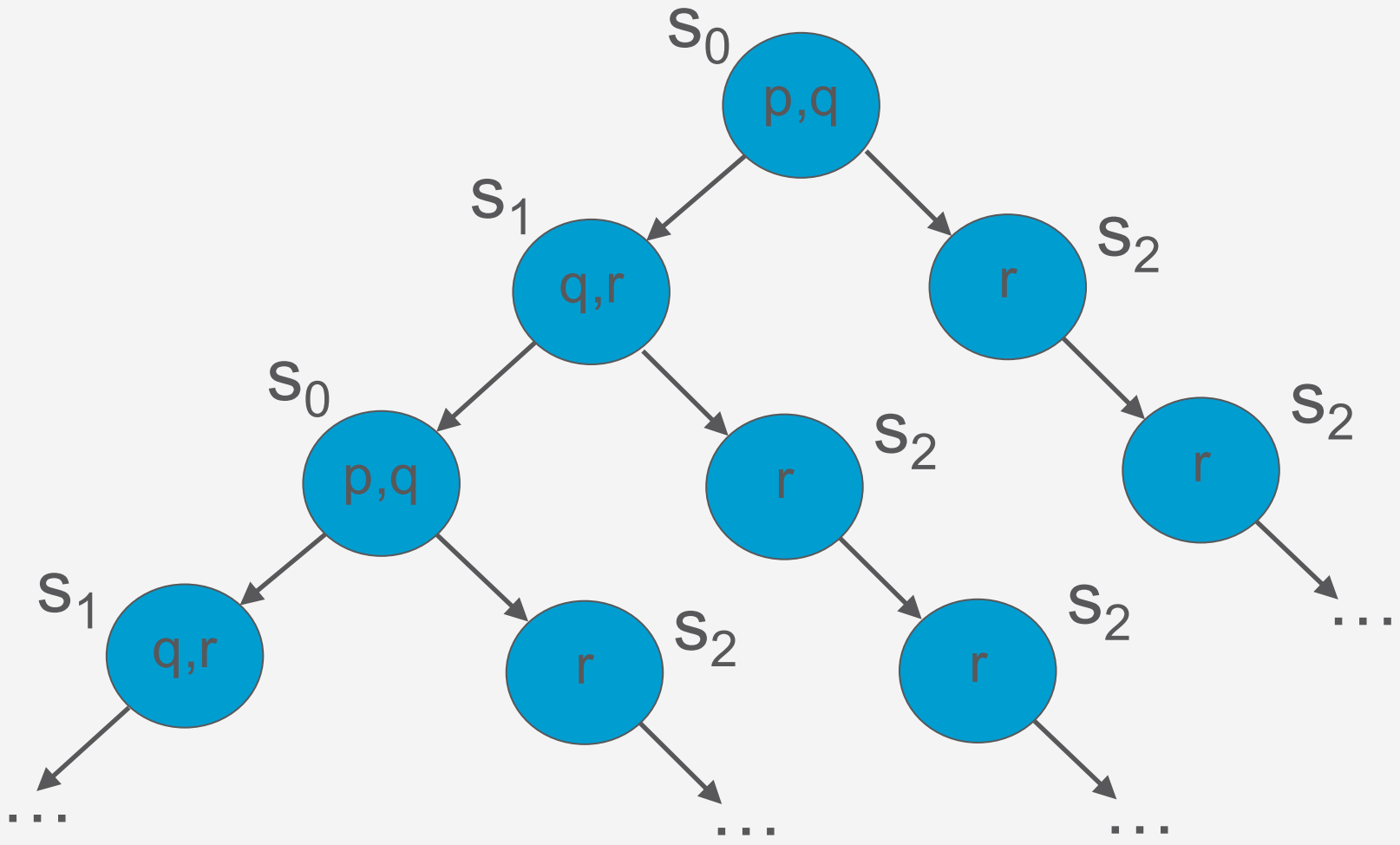
- S is a set of states
- $T \subseteq S \times S$ is a set of transitions
  (often assumed to be total: $\forall s \in S.\ \exists s' \in S.\ (s, s') \in T$)
- $S_0 \subseteq S$ is a set of initial states
- $L\colon S \to 2^{AP}$ is a labeling function
  ($L(s)$ is the set of atomic propositions that are true in s)

Path of a Kripke structure:
$\pi = s_0 s_1 s_2 \ldots$ is a path of (S, T, S_0, L) if $\forall i.\ (s_i, s_{i+1}) \in T$

Transitionssystem: Beispiel

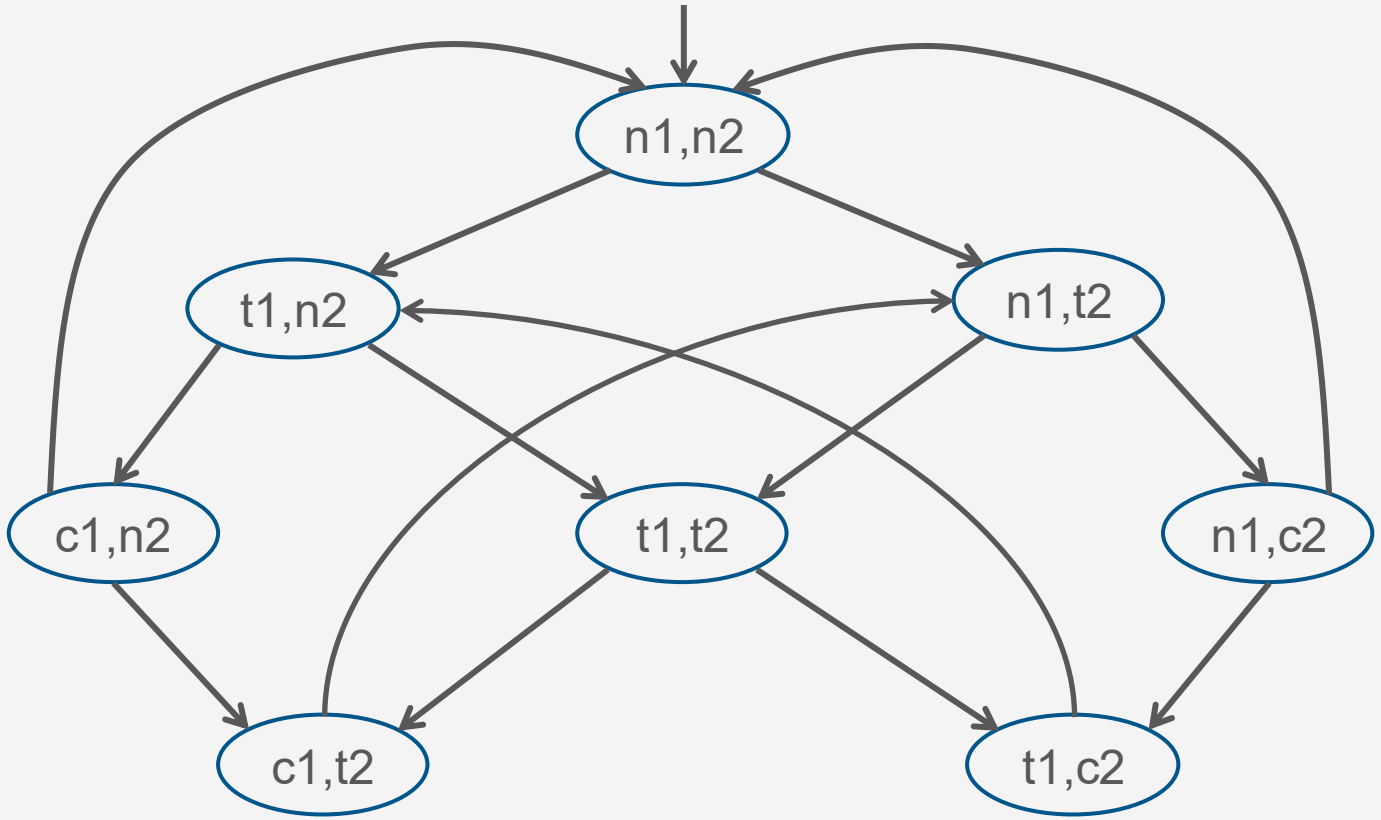


Beispiel: Berechnungsbaum



$M, s_0 \models E\,[(p \wedge q)\ U\ r]$, $M, s_0 \models A\,[p\ U\ r]$

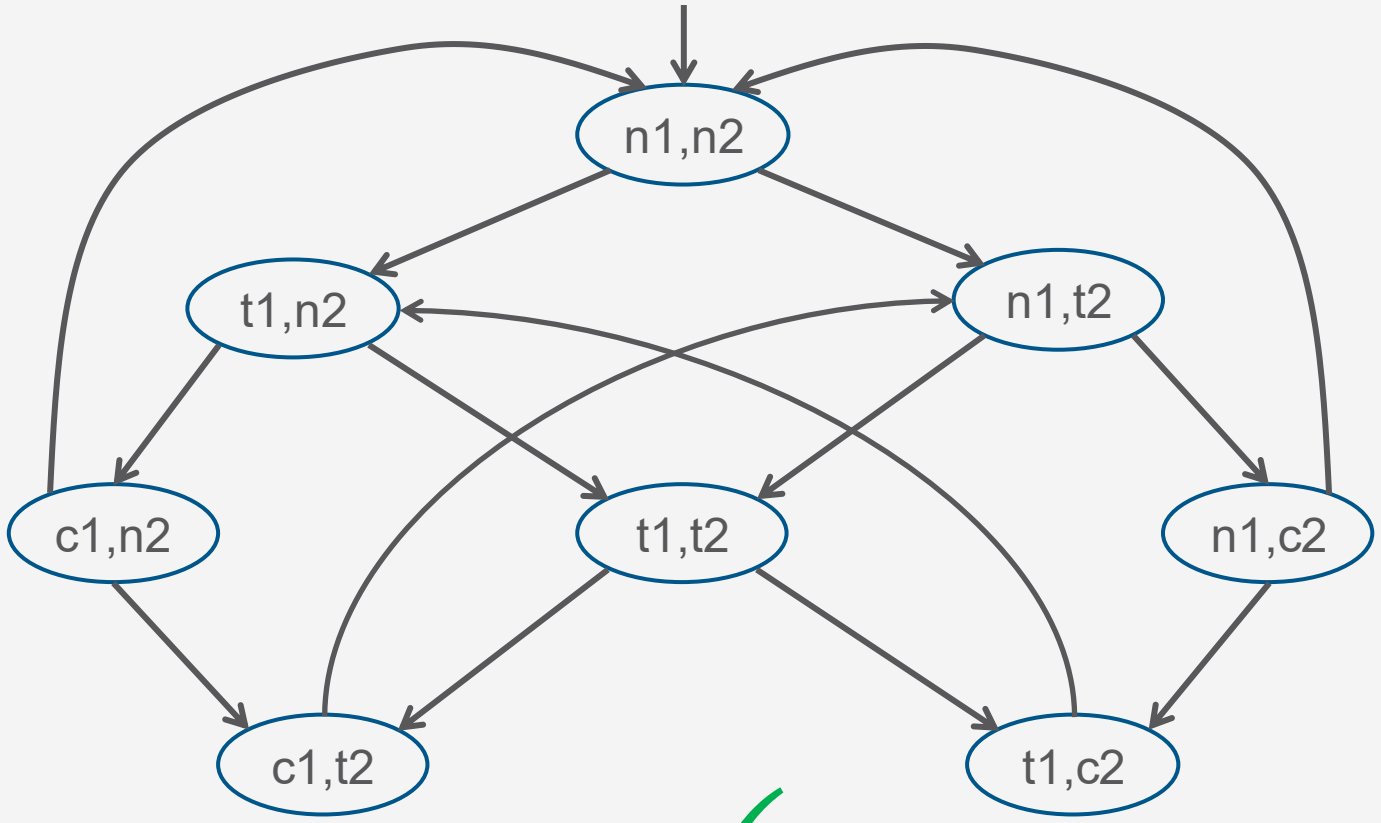
Model Checking: Beispiel



Anforderungen: $M, s_0 \models AG[n_1 \rightarrow EX\ t_1]$, $M, s_0 \models AG[t_1 \rightarrow AF\ c_1]$

Sind diese Anforderungen erfüllt?

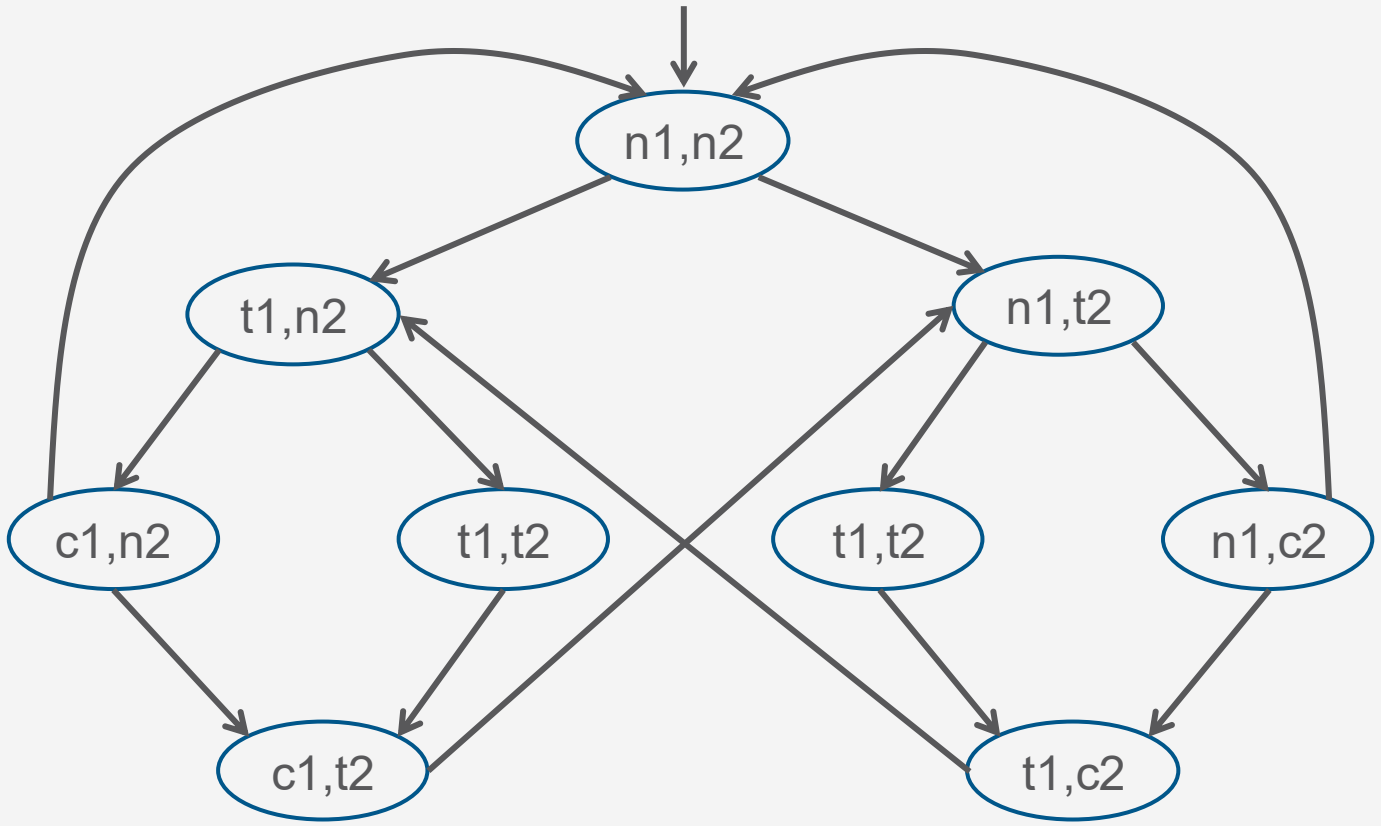
Model Checking: Beispiel



Anforderungen: $M, s_0 \models AG[n_1 \rightarrow EX\ t_1]$ ✓, $M, s_0 \models AG[t_1 \rightarrow AF\ c_1]$ ✗

Sind diese Anforderungen erfüllt?

Model Checking: Beispiel



Anforderungen: $M, s_0 \models AG[n_1 \rightarrow EX\ t_1]$ ✓, $M, s_0 \models AG[t_1 \rightarrow AF\ c_1]$ ✓

Überblick

- Einführung
- Vorüberlegungen: Eigenschaften von CTL
- Labeling-Algorithmus für CTL
- Model Checking als Fixpunktberechnung
- BDD-based Model Checking
- Zusammenfassung

Eigenschaften von CTL (1)

- Eine Menge von temporallogischen Verknüpfungen ist funktional vollständig genau dann wenn es mindestens ein Element aus {AX,EX}, mindestens ein Element aus {AF, EG, AU} und EU enthält.
- Beispiel: {¬, ∧, EX, AF, EU} ist eine funktional vollständige Operatormenge (*functionally complete operator set*)
- Alle anderen Operatoren können daraus konstruiert werden.

Wie können wir die anderen Operatoren konstruieren?

Eigenschaften von CTL (2)

- $\{\neg, \wedge, EX, AF, EU\}$ ist eine funktional vollständige Operatormenge
- Konstruktion der anderen Operatoren:

$$\varphi \vee \psi \equiv \neg(\neg\varphi \wedge \neg\psi)$$

$$AX\ \varphi \equiv \neg\ EX\ \neg\varphi$$

$$EG\ \varphi \equiv \neg\ AF\ \neg\varphi$$

$$AG\ \varphi \equiv \neg\ EF\ \neg\varphi$$

$$EF\ \varphi \equiv E[\mathit{true}\ U\ \varphi]$$

$$A\ [\varphi\ U\ \psi] \equiv \neg\ (E[\neg\psi\ U\ (\neg\varphi \wedge \neg\psi)] \vee EG\ \neg\psi)$$

Eigenschaften von CTL (3)

- AF, EG, und EU können rekursiv definiert werden mit AX und EX

$$AF\ \varphi \equiv \varphi \vee AX\ AF\ \varphi$$

Wie können EG und EU rekursiv definiert werden?

Eigenschaften von CTL (3)

- AF, EG, und EU können rekursiv definiert werden mit AX und EX

$$AF\ \varphi \equiv \varphi \vee AX\ AF\ \varphi$$

Wie können EG und EU rekursiv definiert werden?

$$EG\ \varphi \equiv \varphi \wedge EX\ EG\ \varphi$$

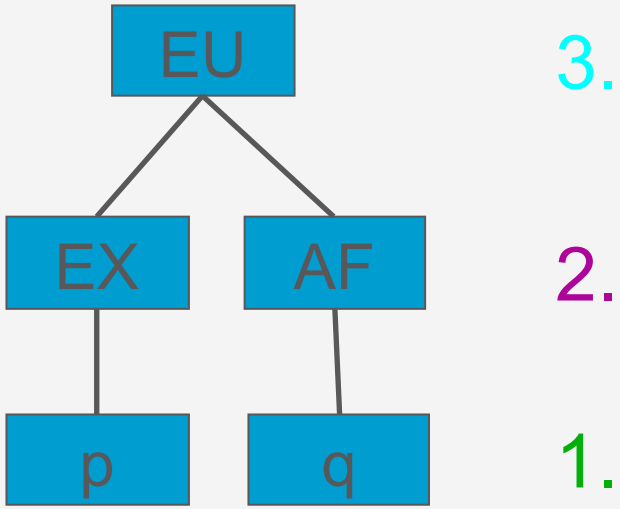
$$E[\varphi\ U\ \psi] \equiv \psi \vee (\varphi \wedge EX\ E[\varphi\ U\ \psi])$$

Überblick

- Einführung
- Vorüberlegungen: Eigenschaften von CTL
- Labeling-Algorithmus für CTL
- Model Checking als Fixpunktberechnung
- BDD-based Model Checking
- Zusammenfassung

Labeling Algorithmus

- Ziel: Bestimme alle Zustände s, die φ erfüllen
- Vorab: Übersetze φ, so dass nur EX, AF, EU, ∧, ¬, ⊥ verwendet werden
- Ansatz: Markiere Zustände mit Teilformeln von φ, beginnend mit der innersten Teilformel
- E[(EX p) U (AF q)]:



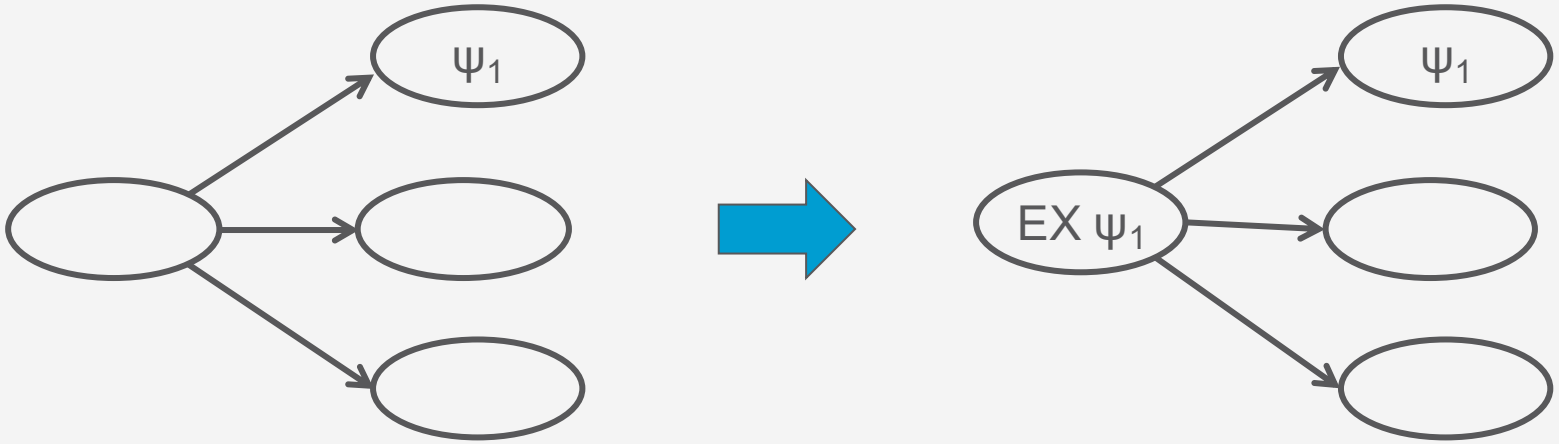
Labeling Algorithmus

- Ziel: Bestimme alle Zustände s, die φ erfüllen
- Vorab: Übersetze φ, so dass nur EX, AF, EU, $\wedge$, $\neg$, $\perp$ verwendet werden
- Ansatz: Markiere Zustände mit Teilformeln von φ, beginnend mit der innersten Teilformel
- für jede Teilformel
  1. p: markiere s mit p falls $p \in L(S)$
  2. $\psi_1 \wedge \psi_2$: markiere s mit $\psi_1 \wedge \psi_2$ falls s mit ψ_1 und ψ_2 markiert ist
  3. $\neg\psi$: markiere s mit $\neg\ \psi$ falls s nicht mit ψ markiert ist

Labeling Algorithmus

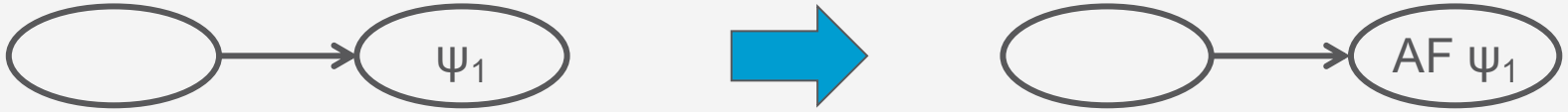
4. EX ψ_1

   Markiere jeden Zustand s mit EX ψ_1, falls einer seiner Folgezustände ψ_1 markiert ist

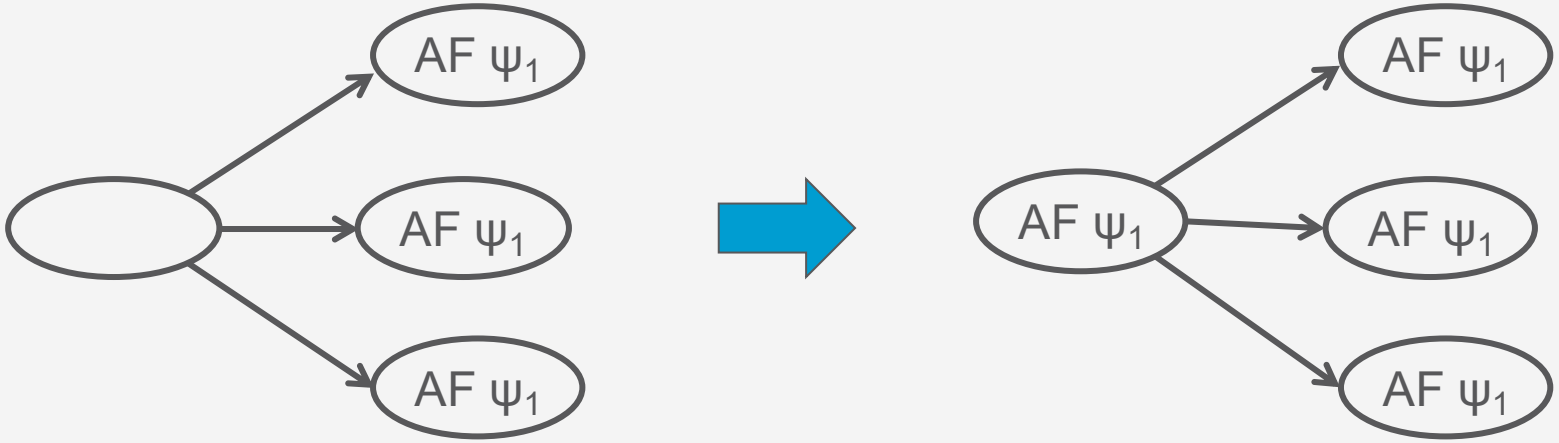


Labeling Algorithmus

5. AF ψ_1
   a. falls ein Zustand s mit ψ_1 markiert ist, markiere s mit AF ψ_1



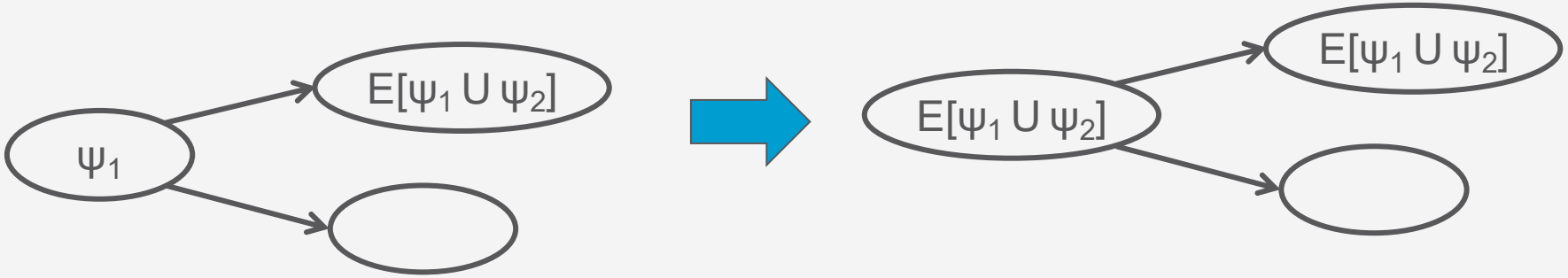
   b. Wiederhole, bis sich nichts mehr ändert:
      falls alle Folgezustände von Zustand s mit AFψ_1 markiert sind, markiere s mit AFψ_1



Labeling Algorithmus

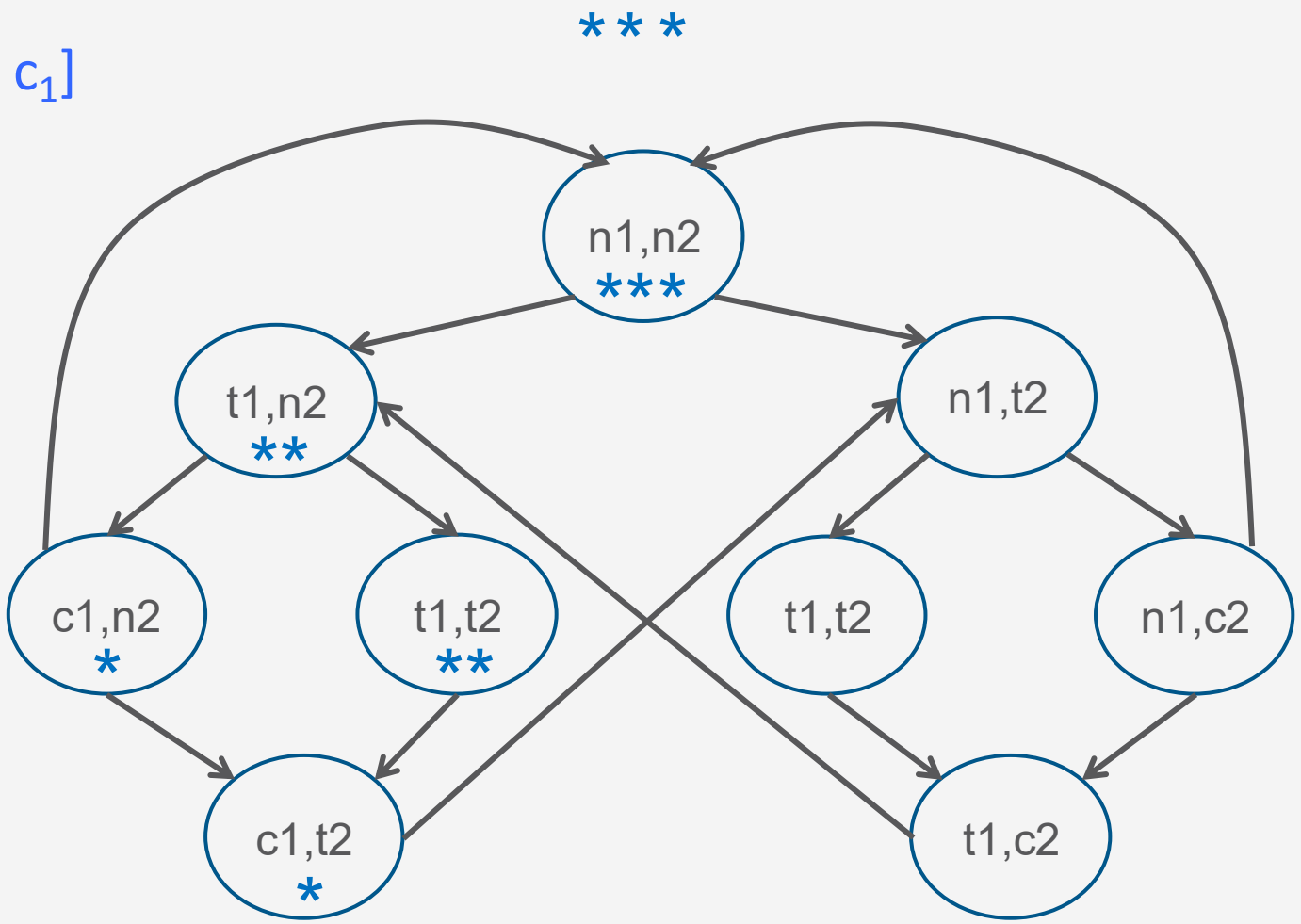
6. $E[\psi_1 \, U \, \psi_2]$
   a. falls ein Zustand s mit ψ_2 markiert ist, markiere s mit $E[\psi_1 \, U \, \psi_2]$

   b. Wiederhole, bis sich nichts mehr ändert:
      markiere Zustand s mit $E[\psi_1 \, U \, \psi_2]$, falls s mit ψ_1 markiert ist und einer der Folgezustände $E[\psi_1 \, U \, \psi_2]$ markiert ist



Beispiel: Labeling Algorithmus

- Beispiel: $E[\neg c_2 \; U \; c_1]$

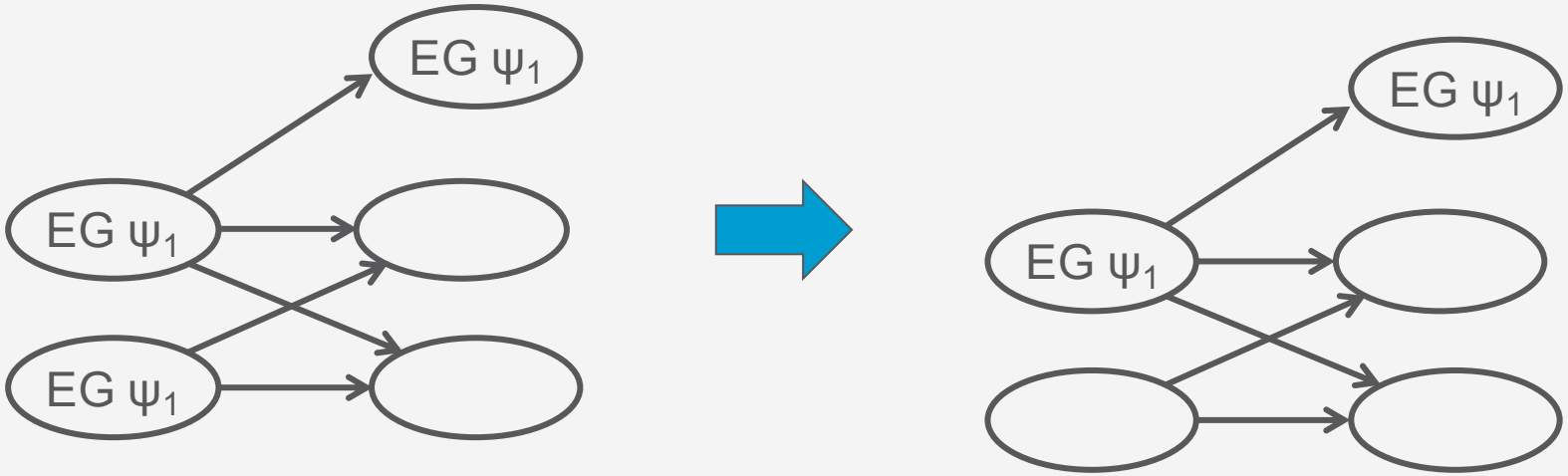


Labeling Algorithmus: Alternative zu AF

EG ψ_1

a. Markiere alle Zustände s mit Markierung ψ_1 mit EG ψ_1

b. Wiederhole, bis sich nichts mehr ändert (Fixpunktberechnung):
lösche Markierung EG ψ_1, falls keiner der Folgezustände mit EG ψ_1 markiert ist



Zusammenfassung: Labeling-Algorithmen

- {¬, ∧, EX, AF, EU} und {¬, ∧, EX, EG, EU} sind jeweils funktional vollständige Operatormengen (alle anderen Operatoren können daraus konstruiert werden).
- Unter Verwendung von funktional vollständigen Operatormengen können wir Labeling Algorithmen für CTL kompakt definieren.
- Ein Labeling Algorithmus für CTL berechnet alle Zustände, die eine gegebene CTL Formel erfüllen.
- Wenn der Initialzustand in der Menge von Zuständen enthalten ist, die eine gegebene Formel erfüllen, erfüllt auch das Gesamtsystem die Formel.

Überblick

- Einführung
- Vorüberlegungen: Eigenschaften von CTL
- Labeling-Algorithmus für CTL
- Model Checking als Fixpunktberechnung
- BDD-based Model Checking
- Zusammenfassung

Prädikat-Transformer

- gegeben: Kripke-Struktur M = (S,S_0,T,L)
- Potenzmenge der Zustände $\mathcal{P}(S)$ bildet einen vollständigen Verband (ausgestattet mit Vereinigung ∪ und Schnittmenge ∩ sowie der Verbandsordnung ⊆)
- Ein Prädikat über Zuständen (z.B. Zustände, in denen p gilt), kann als Teilmenge von $\mathcal{P}(S)$ aufgefasst werden.
- Ein Prädikat-Transformer ist eine Funktion $\tau: \mathcal{P}(S) \to \mathcal{P}(S)$.

Fixpunkte

- Sei $\tau: \mathcal{P}(S) \to \mathcal{P}(S)$ ein Prädikat-Transformer
  - τ ist monoton, wenn aus $P \subseteq Q$ folgt, dass $\tau(P) \subseteq \tau(Q)$
  - τ ist ∪-stetig, falls aus $P_1 \subseteq P_2 \subseteq \ldots$ folgt, dass $\tau(\bigcup_i P_i) = \bigcup_i \tau(P_i)$
  - τ ist ∩-stetig, falls aus $P_1 \supseteq P_2 \supseteq \ldots$ folgt, dass $\tau(\bigcap_i P_i) = \bigcap_i \tau(P_i)$
- eine Menge $S' \subseteq S$ ist ein Fixpunkt von τ, falls $\tau(S') = S'$
- jeder monotone Prädikat-Transformer besitzt einen kleinsten Fixpunkt ($\mu Z.\, \tau(Z)$) und größten Fixpunkt ($\nu Z.\, \tau(Z)$)

Fixpunktberechnung

- Satz (Tarski / Kleene):
  1. Falls τ monoton ist, so ist der kleinste Fixpunkt $\mu Z.\tau(Z) = \bigcap\{Z \mid \tau(Z) = Z\}$ und der größte Fixpunkt $\nu Z.\tau(Z) = \bigcup\{Z \mid \tau(Z) = Z\}$
  2. Falls τ monoton und $\cup$-stetig ist, so ist $\mu Z.\tau(Z) = \bigcup_i \tau^i(\emptyset)$.
  3. Falls τ monoton und $\cap$-stetig ist, so ist $\nu Z.\tau(Z) = \bigcap_i \tau^i(Z)$.
- Falls S endlich und τ monoton ist, so ist τ immer auch $\cup$-stetig und $\cap$-stetig.

➢ für endliche Mengen können Fixpunkte mit 2. und 3. berechnet werden

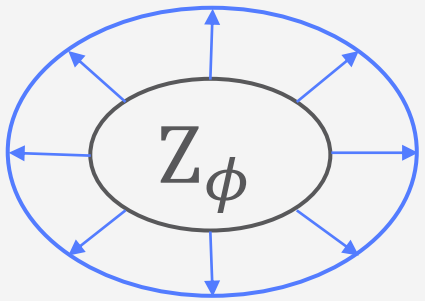
- Berechnung der Fixpunkte

$$\mu Z.\tau(Z): \emptyset, \tau(\emptyset), \tau^2(\emptyset), \tau^3(\emptyset), \ldots$$
$$\nu Z.\tau(Z): S, \tau(S), \tau^2(S), \tau^3(S), \ldots$$

Sobald $\tau^{i+1}(Z) = \tau^i(Z)$: Fixpunkt gefunden!

Fixpunktberechnung: EF ϕ

Z := Z_ϕ // alle Zustände, die ϕ erfüllen
do
  Z := Z $\vee$ EX Z // inkrementelle Erweiterung von Z
  // Prädikat-Transformer $\tau(Z) = \phi \vee EXZ$
  until nothing changes // Fixpunkt erreicht ($Z_i \equiv Z_{i-1}$)
return Z

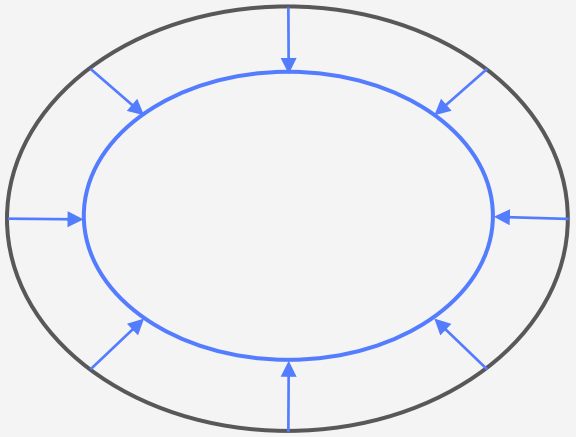


- Starte mit allen Zuständen, die ϕ erfüllen (Z_ϕ)
- In jedem Schritt: erweitere mit allen Vorzuständen, also allen Zuständen, wo mindestens ein Folgezustand in Z enthalten ist (EX Z)

Kurznotation: $EF\phi = \mu Z.\ \phi \vee EX\ Z$ (kleinster Fixpunkt)

Fixpunktberechnung: EG ϕ

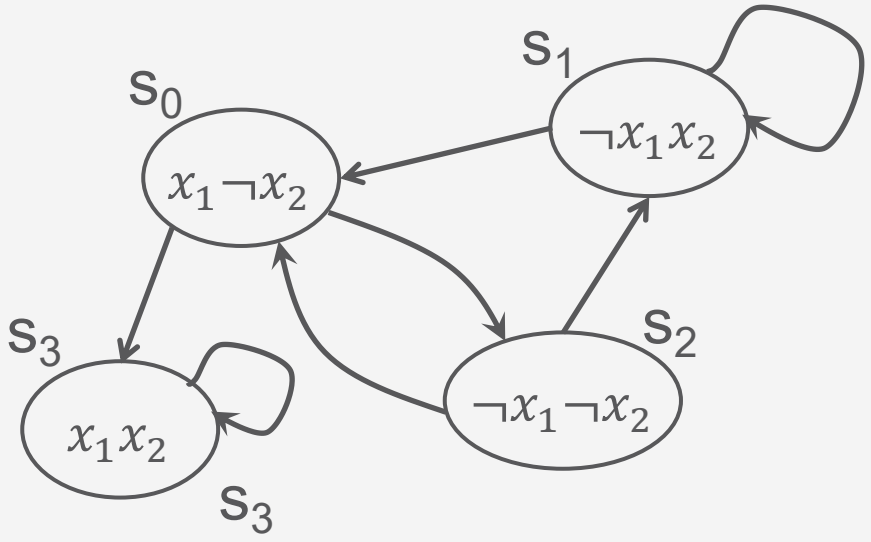
Z := Z_ϕ // alle Zustände, die ϕ erfüllen
do
  Z := Z $\wedge$ EX Z // inkrementelle Einschränkung von Z
  until nothing changes // Fixpunkt erreicht ($Z_i \equiv Z_{i-1}$)
return Z



- Starte mit allen Zuständen, die ϕ erfüllen (Z_ϕ)
- In jedem Schritt: schränke Z ein auf alle Zustände, in denen mindestens ein Folgezustand ebenfalls in Z liegt (EX Z)

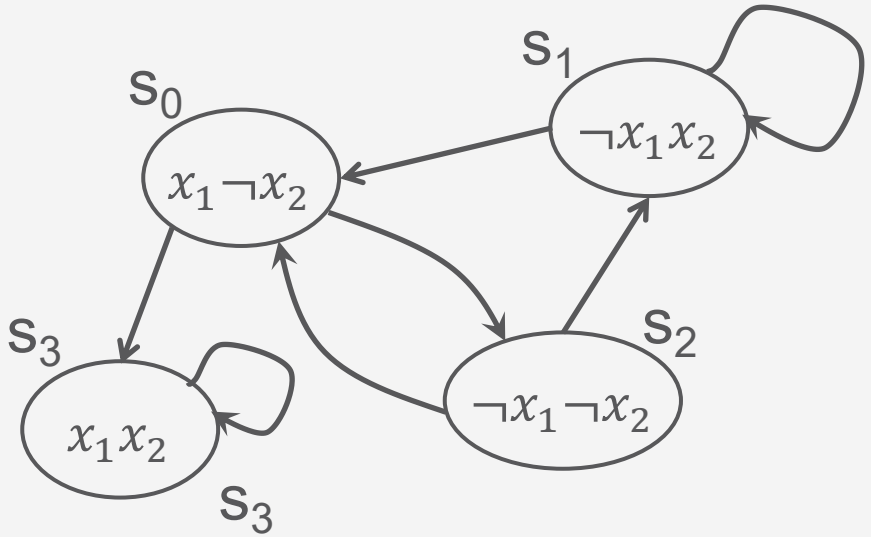
Kurznotation: $EG\phi = \nu Z.\, \phi \wedge EXZ$ (größter Fixpunkt)

Fixpunktberechnung: Beispiel



$$\boldsymbol{AG\,(x_1 \vee \neg x_2)}\ \textbf{?}$$

Fixpunktberechnung: Beispiel



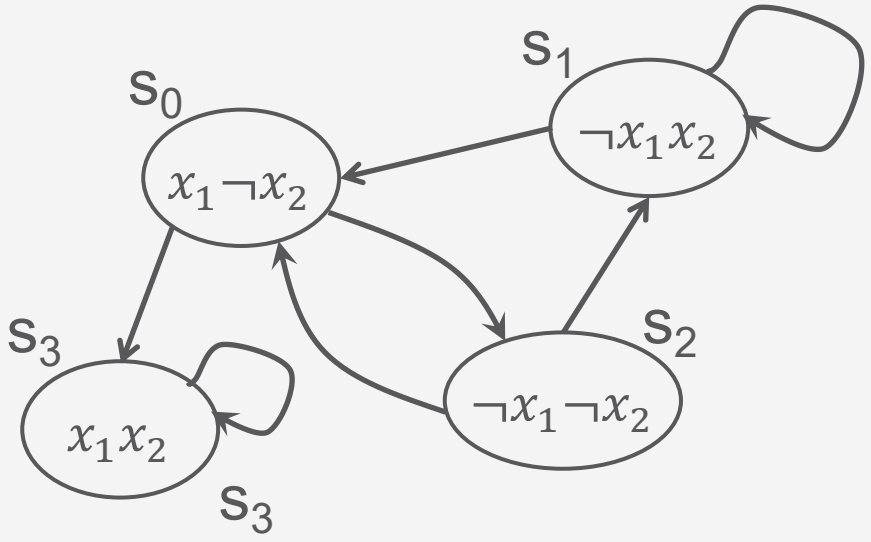
$\boldsymbol{AG\ (x_1 \vee \neg x_2)} \equiv \neg EF(\neg x_1 \wedge x_2)$

⇨ $EF\phi = \mu Z.\ \phi \vee EXZ$ (kleinster Fixpunkt)
mit $\phi = (\neg x_1 \wedge x_2)$

- $Z_0 = Z_\phi = (\neg x_1 \wedge x_2)$ $\{s_1\}$
- $Z_1 = Z_\phi \vee EX\ Z_0 = (\neg x_1 \wedge x_2)$ $\{s_1\} \cup \{s_1, s_2\} = \{s_1, s_2\}$
- $Z_2 = Z_1 \vee EX\ Z_1 = (x_1 \wedge \neg x_2) \vee \neg\ x_1$ $\{s_1, s_2\} \cup \{s_1, s_2, s_0\} = \{s_0, s_1, s_2\}$
- $Z_3 = Z_2$ **Fixpunkt!**

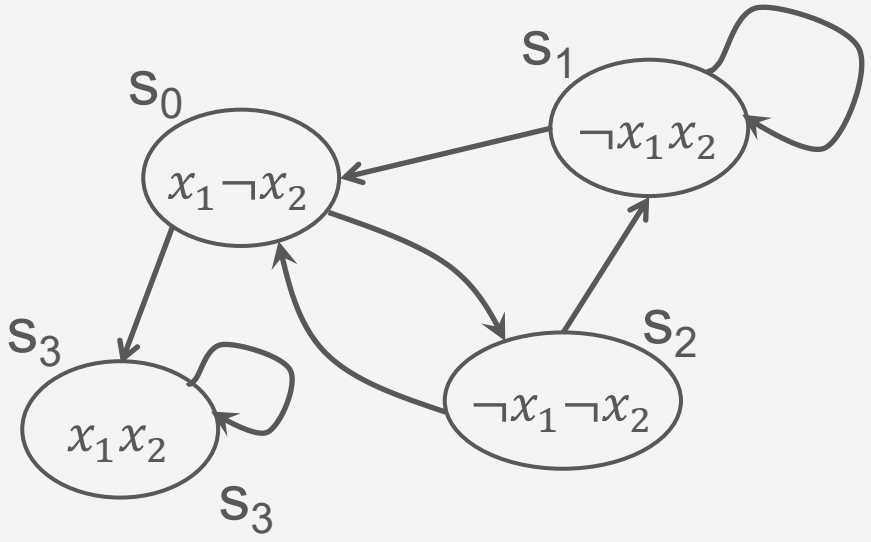
$\mathbf{EF}\ \boldsymbol{\phi} : \{\boldsymbol{s_0}, \boldsymbol{s_1}, \boldsymbol{s_2}\}, \boldsymbol{AG}\ \neg\boldsymbol{\phi} : \{\boldsymbol{s_3}\}$

Fixpunktberechnung: noch ein Beispiel



$$\boldsymbol{EG\,(x_1 \equiv x_2)\ ?}$$

Fixpunktberechnung: noch ein Beispiel



$\boldsymbol{EG\,(x_1 \equiv x_2)}$ **?**

⇨ $EG\phi = \nu Z.\, \phi \wedge EXZ$ (größter Fixpunkt)
mit $\phi = (x_1 \equiv x_2)$

- $Z_0 = (\neg x_1 \wedge \neg x_2) \vee (x_1 \wedge x_2)$ $\quad \{s_2, s_3\}$
- $Z_1 = Z_0 \wedge EX\, Z \;\; = x_1 \wedge x_2$ $\quad \{s_2, s_3\} \wedge \{s_0, s_3\} = \{s_3\}$
- $Z_2 = Z_1$ **Fixpunkt!**

$\mathbf{EG}\ \boldsymbol{\phi} = \{\boldsymbol{s_3}\}$

Recap: Große Kripke-Strukturen

- Problem der Zustandsexplosion

- Typische HW-Designs: $2^{1000000}$ Zustände
- Kripke-Struktur ist i.d.R. exponentiell größer als ein Programm
  - Nebenläufige oder parallele Prozesse (kombinatorische Zustandsexplosion)
  - Instantiierung von Variablen
  - Verschachtelte Prozeduren

Recap: Lösungsansätze

- Konservativ:
  - Darstellung der Kripke-Struktur durch eine spezielle Datenstruktur
  - ➢ Komprimierung der Informationen, z.B. Symbolisches Model Checking

  Idee: BDDs zur Darstellung von Zustandsmengen und der Übergangsfunktion

- Aggressiv:
  - Verwendung von Wissen über die Kripke-Struktur, um von irrelevanten Details zu abstrahieren.
  - ➢ Filterung von Informationen
  - ➢ z.B. Existentielle Abstraktion (Zustände weglassen)

Überblick

- Einführung
- Vorüberlegungen: Eigenschaften von CTL
- Labeling-Algorithmus für CTL
- Model Checking als Fixpunktberechnung
- BDD-based Model Checking
- Zusammenfassung

Symbolisches Model Checking

- Idee: Boolesche Kodierung von Zuständen und Transitionen
- platzsparende Darstellung und effiziente Operationen
- zur Repräsentation von Booleschen Funktionen werden traditionell Binary Decision Diagrams (BDDs) verwendet
- noch heute in vielen industriellen Model Checkern

Relationen als Boolesche Formeln

- Sei Q eine n-äre Relation über {0,1}, dann ist die charakteristische Funktion von Q definiert als:

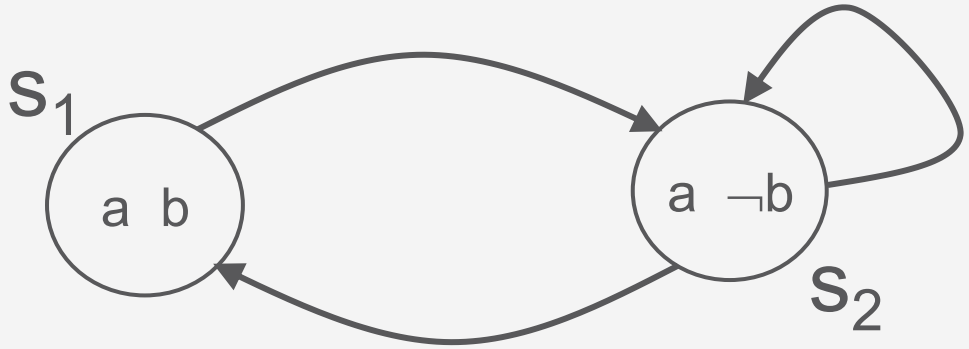
$$f_Q(x_1,\ldots,x_n) = 1 \quad \textit{iff} \quad Q(x_1,\ldots,x_n)$$

⇨ f_Q kann durch eine Boolesche Formel repräsentiert werden

Generalisierung

- Sei Q eine Relation über der *endlichen* Domäne D mit 2^m Elementen ($m > 1$),
- Sei $\phi : \{0,1\}^m \rightarrow D$ eine bijektive Funktion, die den Elementen aus D Boolesche Vektoren der Länge m zuordnet
- Dann kann die charakteristische Funktion von $Q'(\mathbf{x}_1,\ldots,\mathbf{x}_n) = Q(\phi(\mathbf{x}_1),\ldots,\phi(\mathbf{x}_n))$ (mit $\mathbf{x}_i = x_{i,1}, x_{i,2}, \ldots, x_{i,m}$) als eine Boolesche Funktion repräsentiert werden.

Kripke-Strukturen als Boolesche Formeln

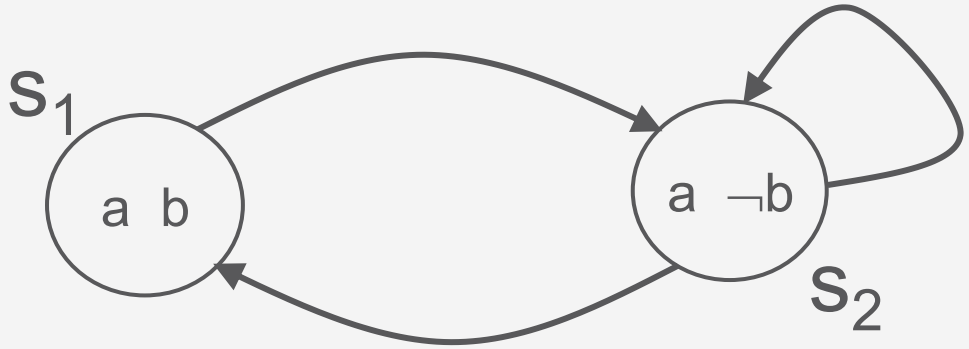


- Zustände: $s_1 = a \wedge b$, $s_2 = a \wedge \neg b$
- Transition von s_1 nach s_2: $a \wedge b \wedge a' \wedge \neg b'$
- Transitionsrelation (im Allgemeinen)

$$\bigvee_{(s,s')\in R} f_{(s,s')}(x_1, \dots x_n, x_1', \dots, x_n') = \bigvee_{(s,s')\in R} f_{(s)}(x_1, \dots x_n,) \wedge f_{(s')}(x_1', \dots, x_n')$$

- Wie kann die Transitionsrelation des Beispiels kodiert werden?

Kripke-Strukturen als Boolesche Formeln



- Zustände: $s_1 = a \wedge b$, $s_2 = a \wedge \neg b$
- Transition von s_1 nach s_2: $a \wedge b \wedge a' \wedge \neg b'$
- Transitionsrelation (im Allgemeinen)

$$\bigvee_{(s,s')\in R} f_{(s,s')}(x_1, \dots x_n, x_1', \dots, x_n') = \bigvee_{(s,s')\in R} f_{(s)}(x_1, \dots x_n,) \wedge f_{(s')}(x_1', \dots, x_n')$$

- hier: $(a \wedge b \wedge a' \wedge \neg b') \vee (a \wedge \neg b \wedge a' \wedge \neg b') \vee (a \wedge \neg b \wedge a' \wedge b')$

Boolesche Kodierung von Zuständen

- In einer Kripke-Struktur kann ein Zustand als Zuweisung eines Wahrheitswerts zu allen atomaren Aussagen aufgefasst werden.

- Menge Q von Zuständen: $f_Q(x_{1,\ldots,}x_n) = \bigvee f_s(x_{1,\ldots,}\ x_n),\ s \in Q$
- Beispiel: drei Zustandsvariablen $x_{1,}x_{2,}x_3$
  - Q = {000,001,010,011} ➔ $f_Q(x_{1,}x_{2,}x_3) = \ldots = \neg x_1$
  - z.B.: 010 ist der Zustand in dem nur x_2 gilt

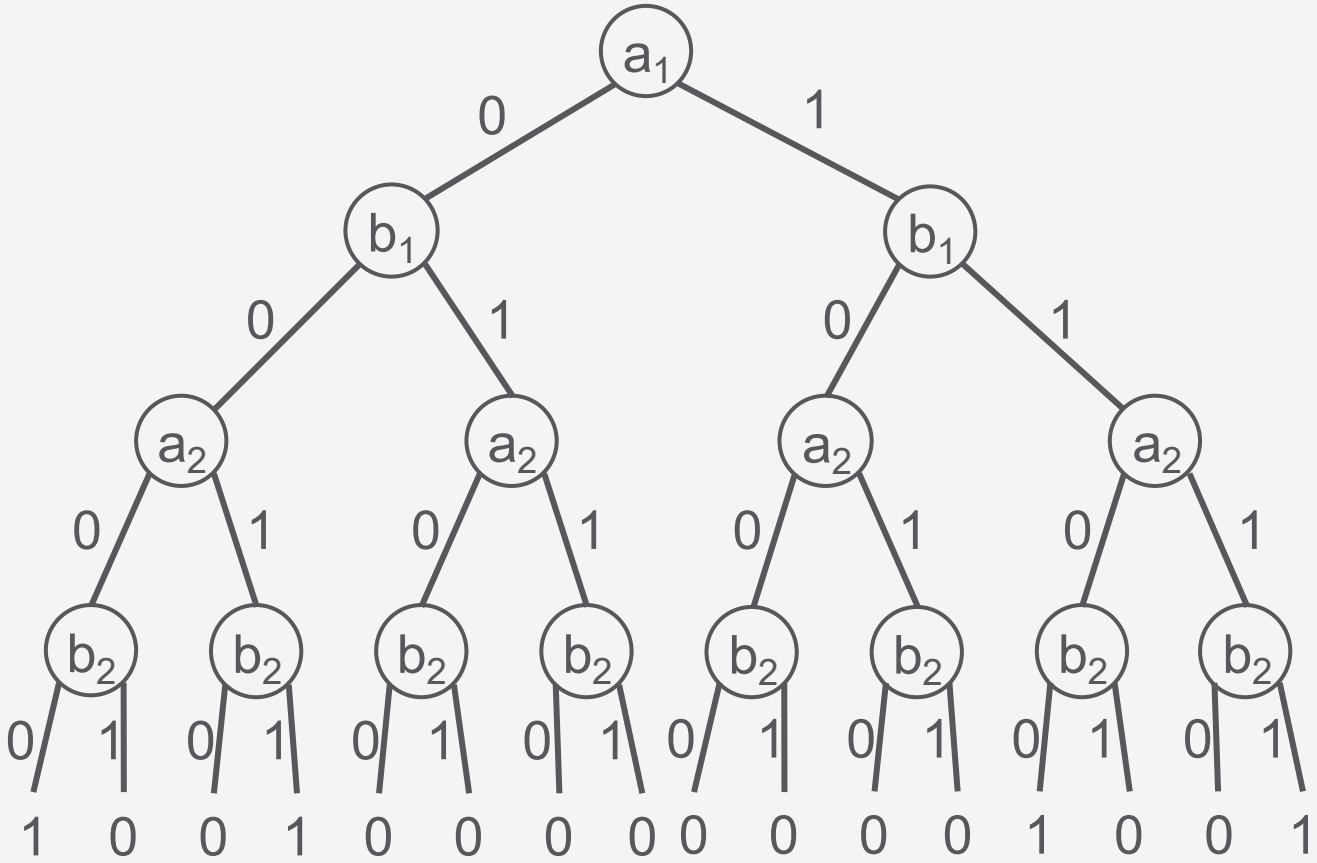
- komplexe (endliche) Datentypen können in eine solche Repräsentation mit Hilfe einer bijektiven Funktion ϕ abgebildet werden

Binary Decision Diagrams (BDDs)

- Symbolische Repräsentation von reaktiven Systemen mit endlichen Zustandsraum basierend auf Booleschen Funktionen
- Kodierung von Kripke-Strukturen
- kompakte Repräsentation von *synchronen* und *asynchronen* Systemen
- effiziente Operationen auf BDDs

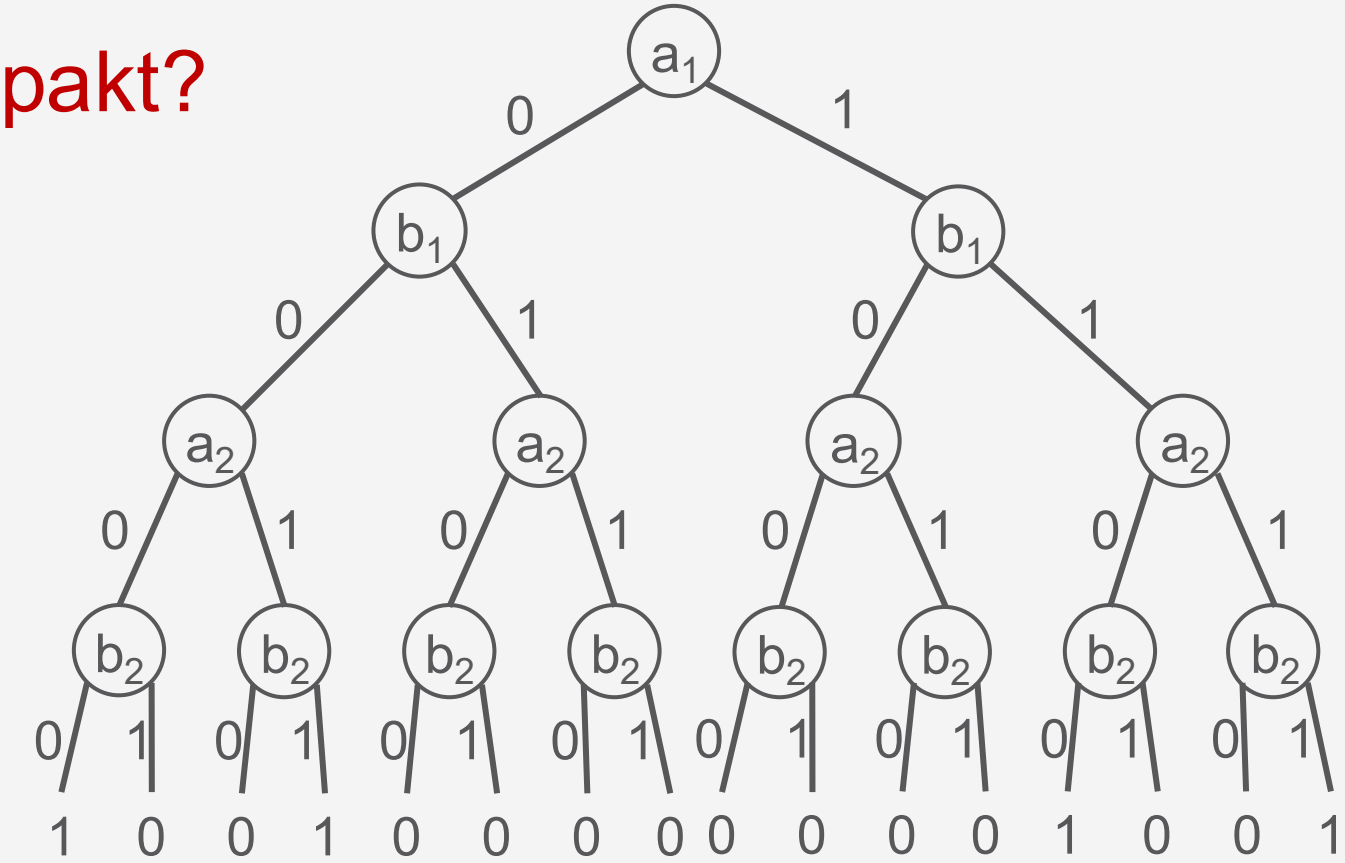
⇨ BDD-basiertes Model Checking

Binary Decision Tree (2-Bit Comparator)



Binary Decision Tree (2-Bit Comparator)

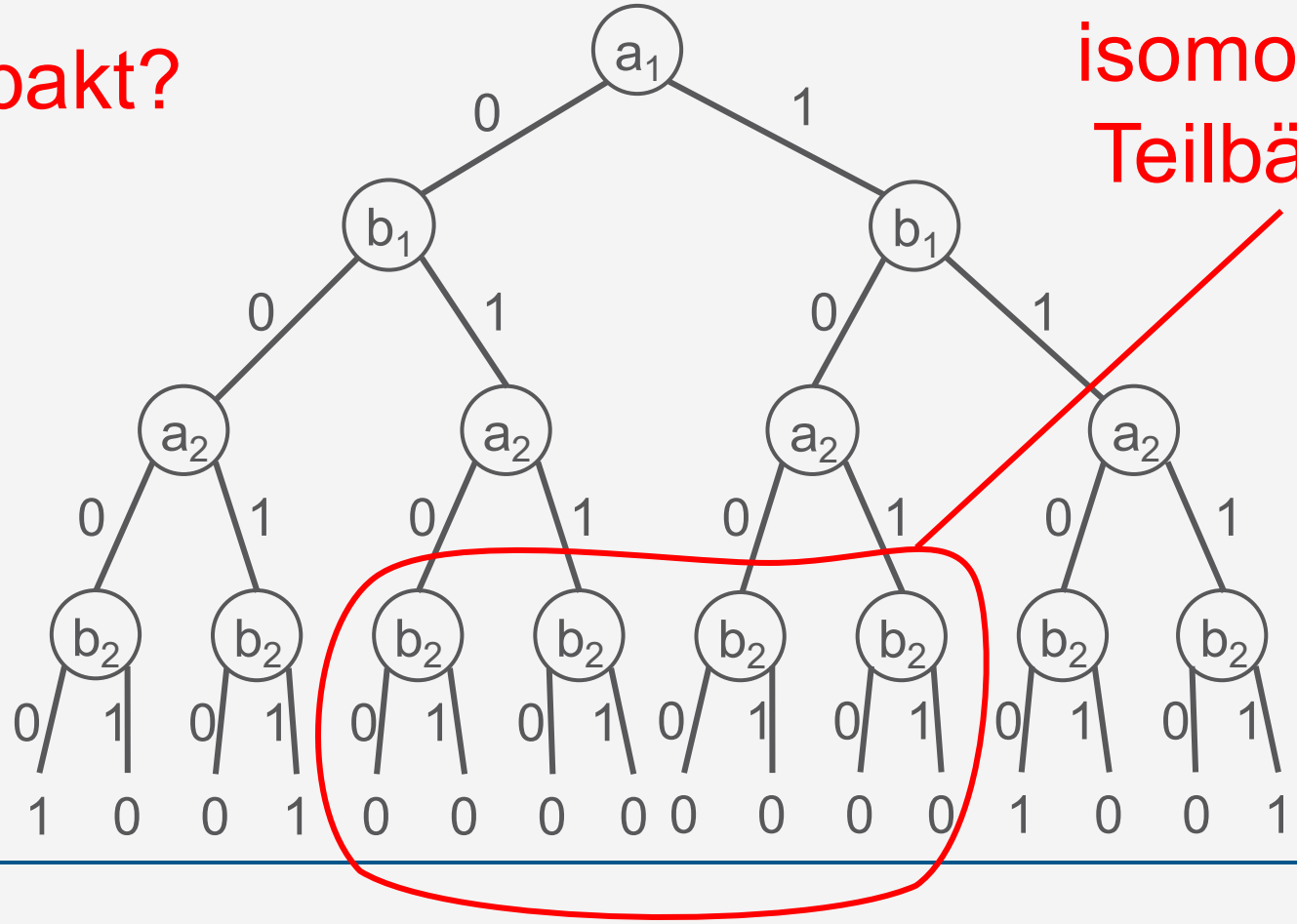
kompakt?



Binary Decision Tree (2-Bit Comparator)

kompakt?

isomorphe Teilbäume

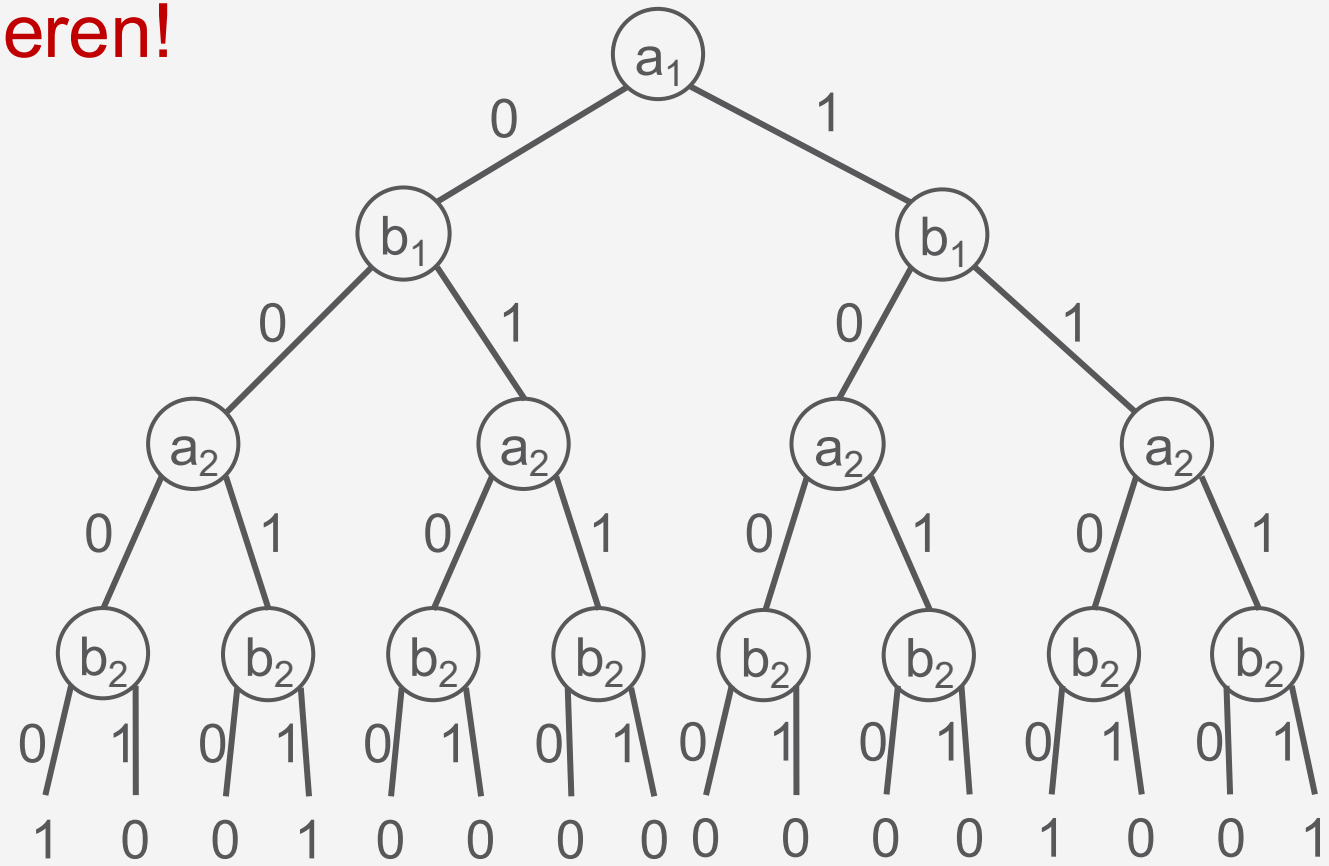


Ordered Binary Decision Diagram (OBDD)

- Aggregation von isomorphen Teilbäumen: gerichteter azyklischer Graph (*directed acyclic graph, DAG*)
  ⇨ Binary Decision Diagram
- kanonische Form:
  - feste Reihenfolge von Variablen entlang jedes Pfades (*ordered BDD*)
  - keine isomorphen Teilbäume und keine redundanten Knoten

Binary Decision Tree (2-Bit Comparator)

Versuchen Sie, diesen Baum zu reduzieren!

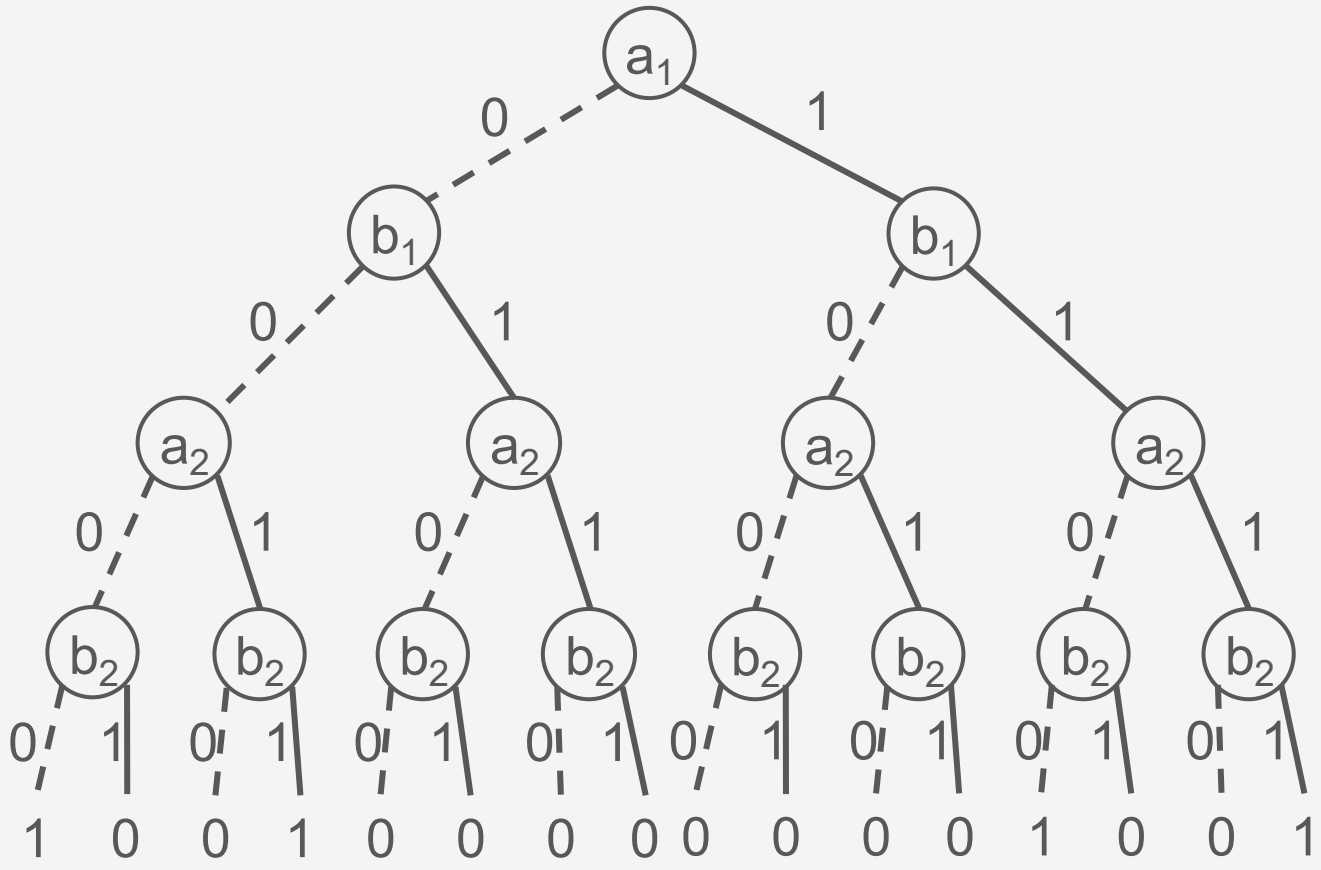


Ordered Binary Decision Diagram (OBDD)

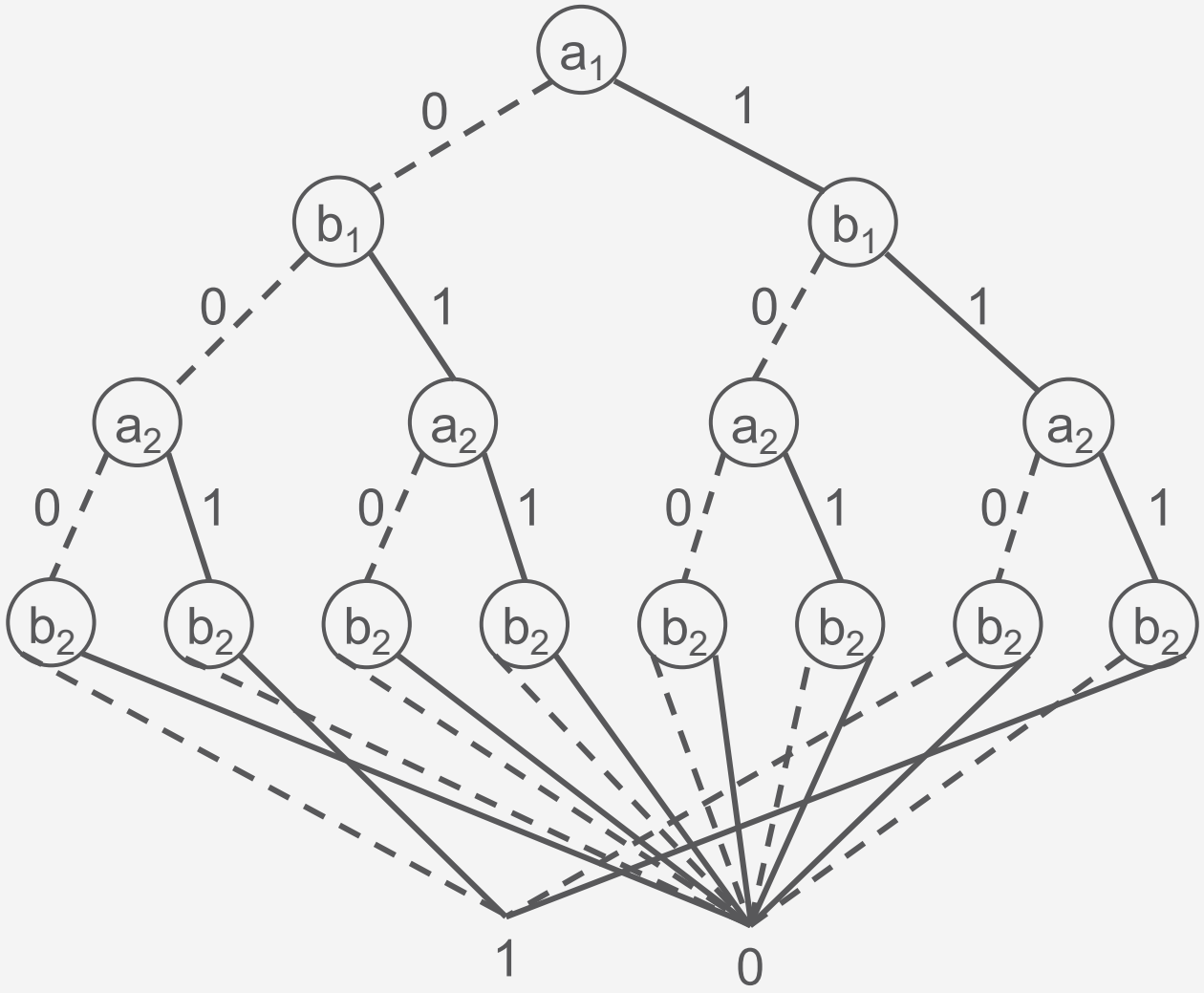
Vom binären Entscheidungsbaum (*binary decision tree*) zum geordneten binären Entscheidungsdiagramm (*ordered binary decision diagram*):

1. Entferne redundante terminale Knoten
2. Entferne redundante nicht-terminale Knoten
3. Entferne redundante Tests

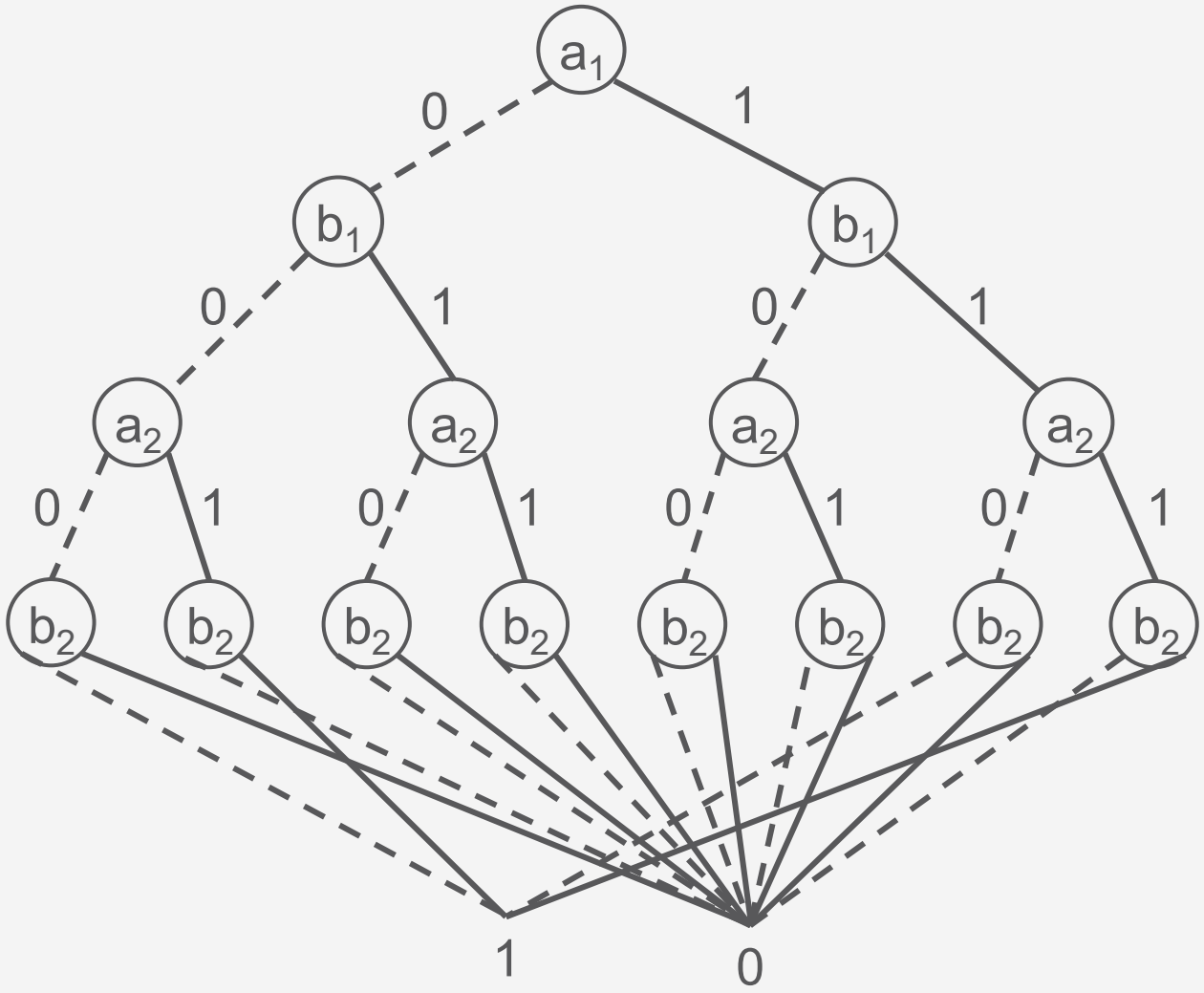
Entfernung redundanter terminaler Knoten



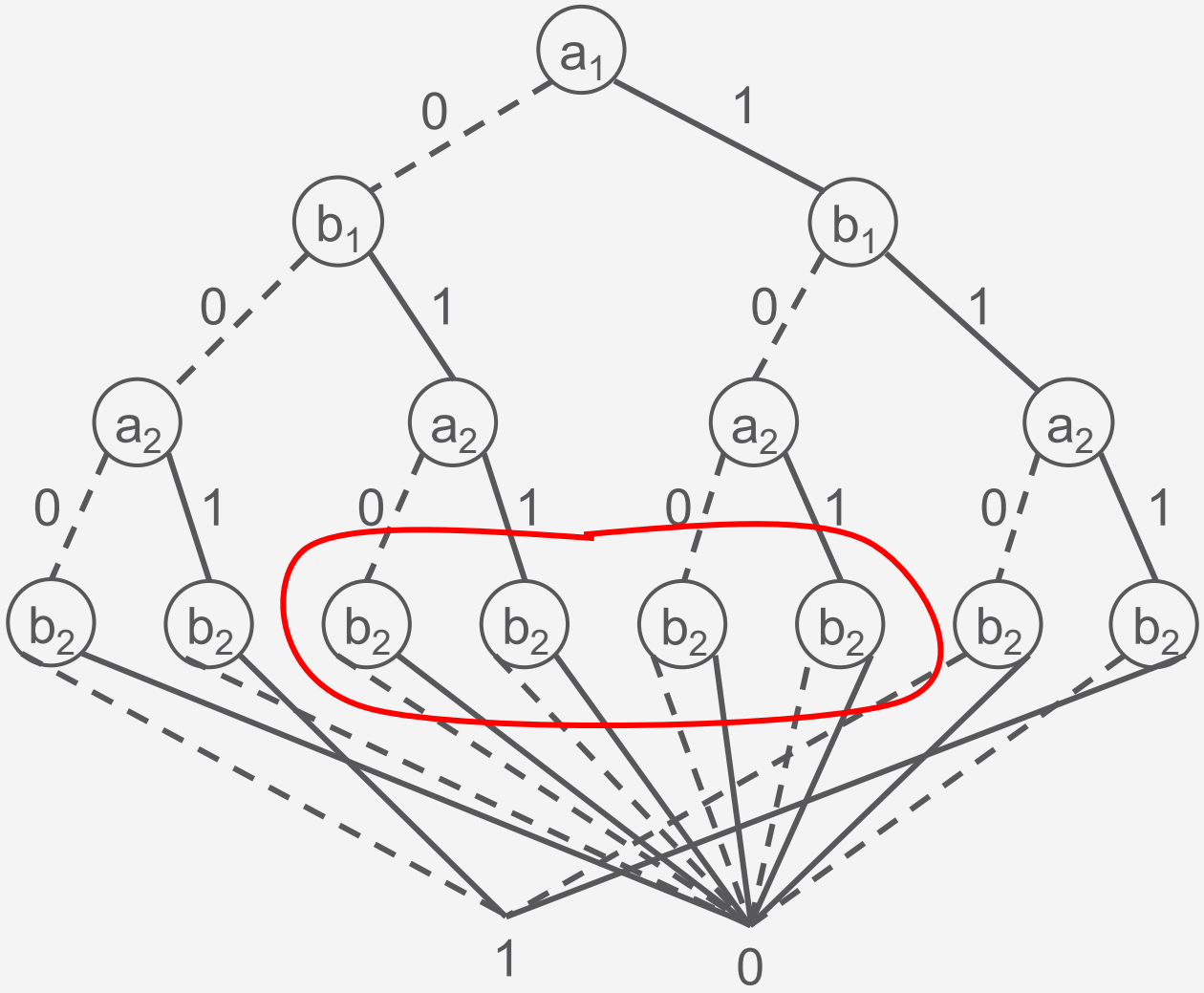
Entfernung redundanter terminaler Knoten



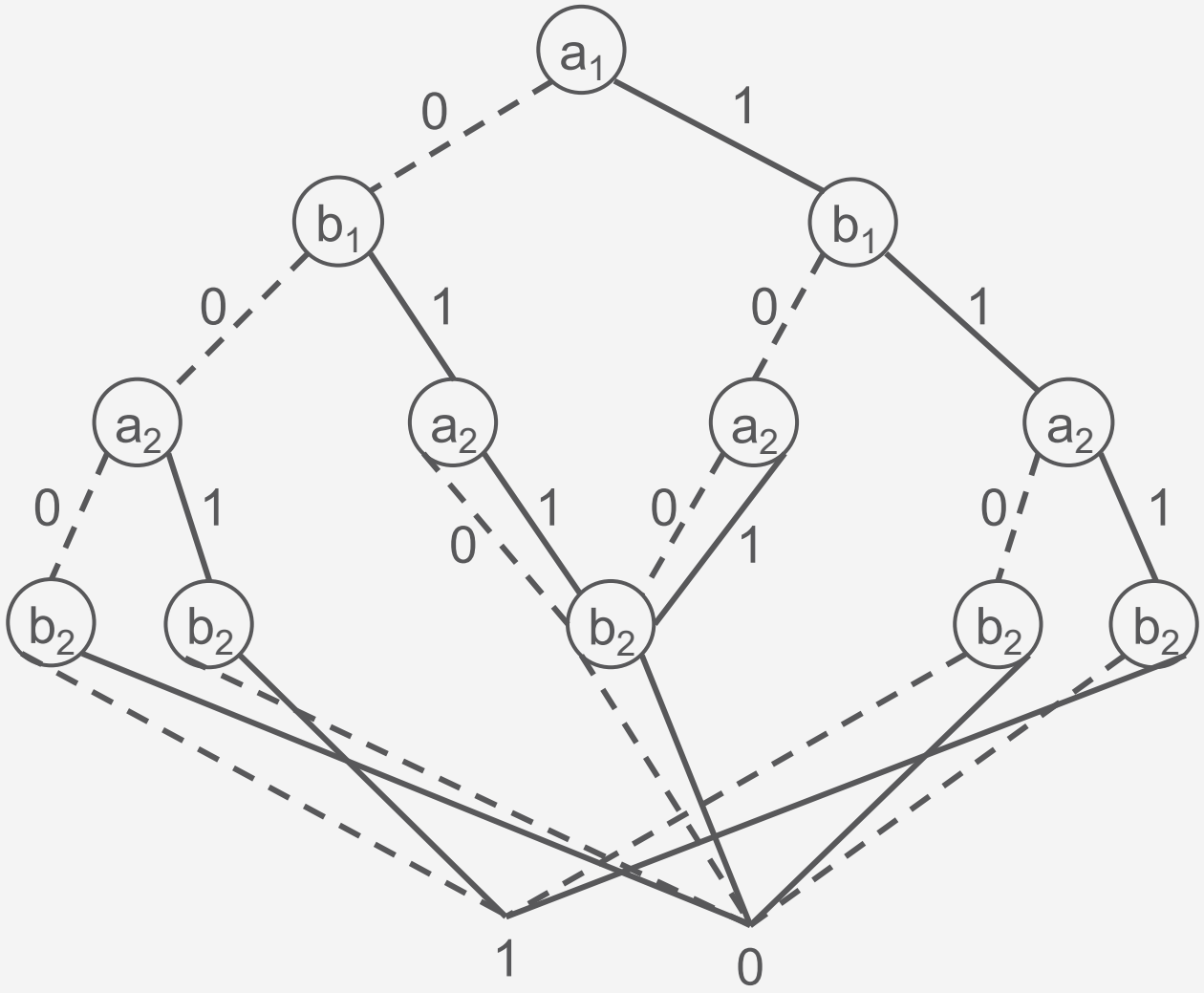
Entfernung redundanter nicht-terminaler Knoten



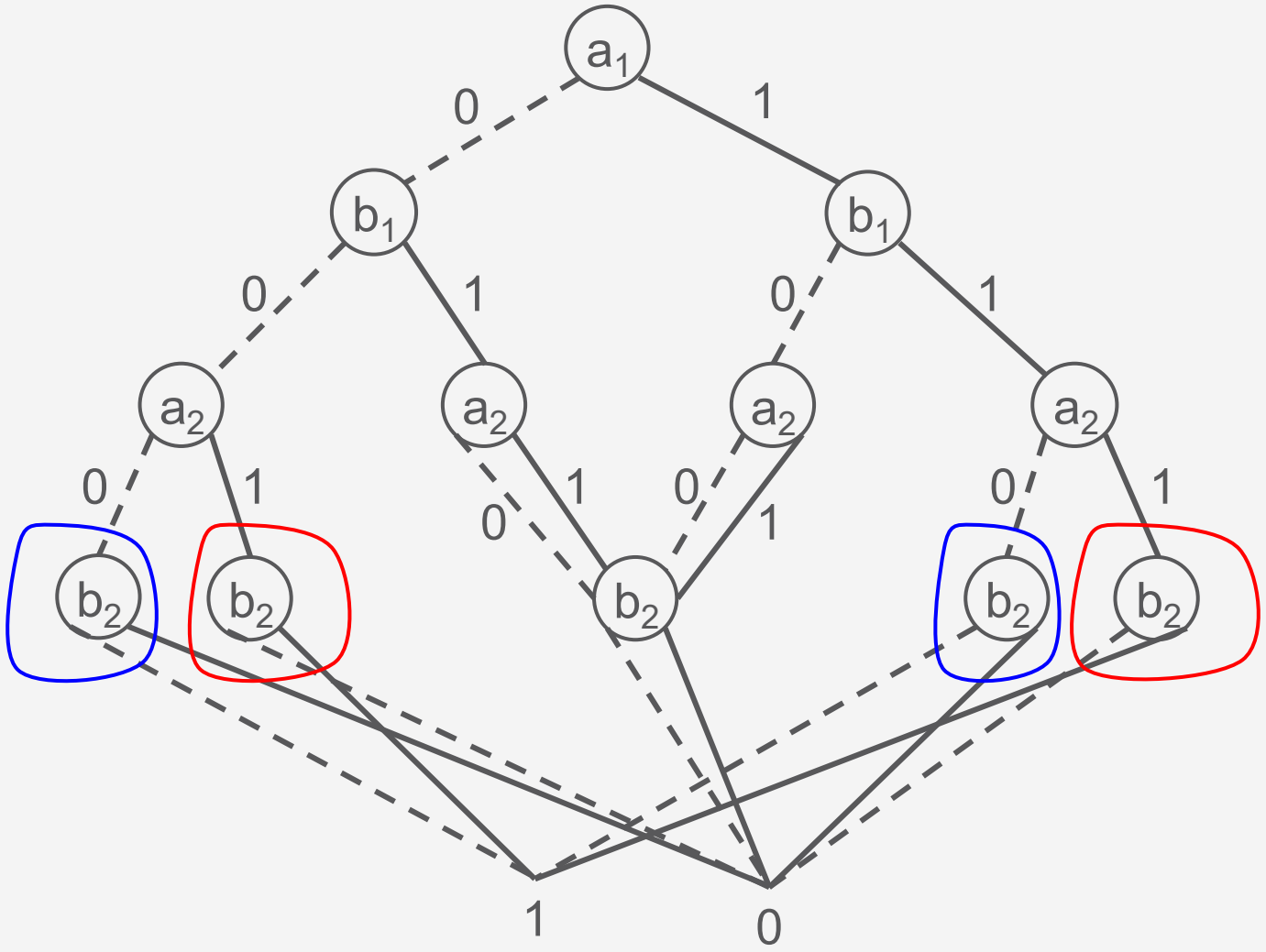
Entfernung redundanter nicht-terminaler Knoten



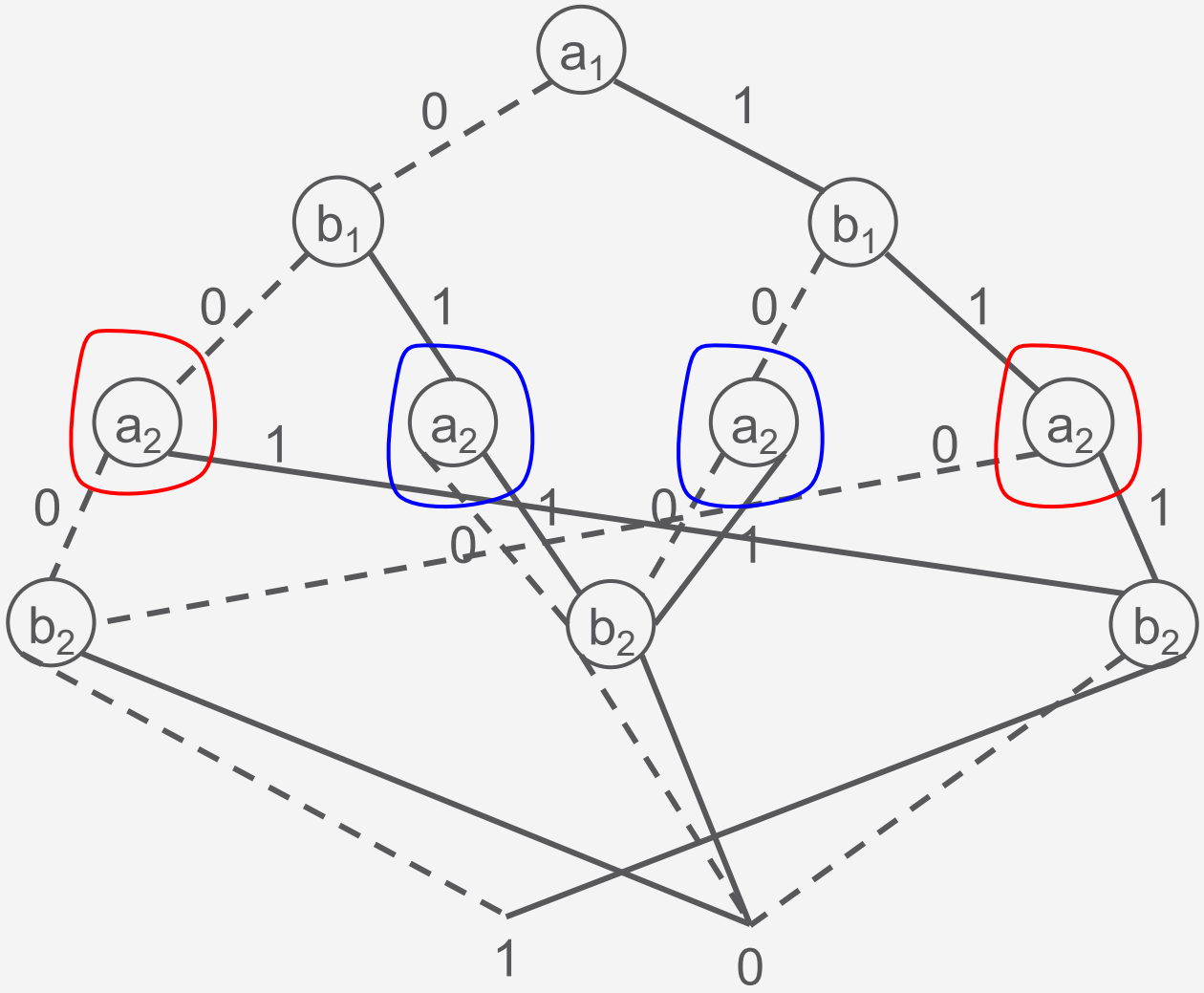
Entfernung redundanter nicht-terminaler Knoten



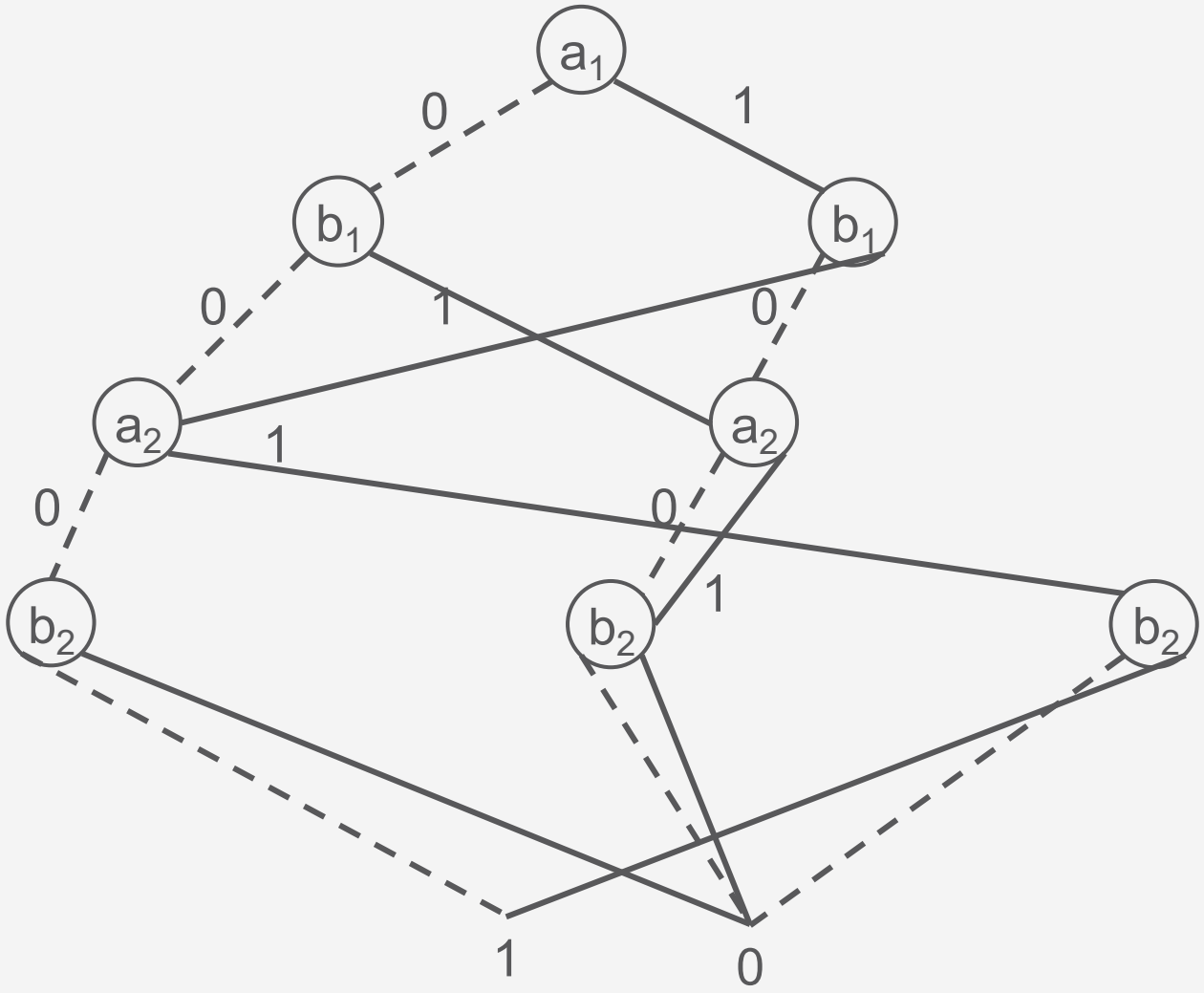
Entfernung redundanter nicht-terminaler Knoten



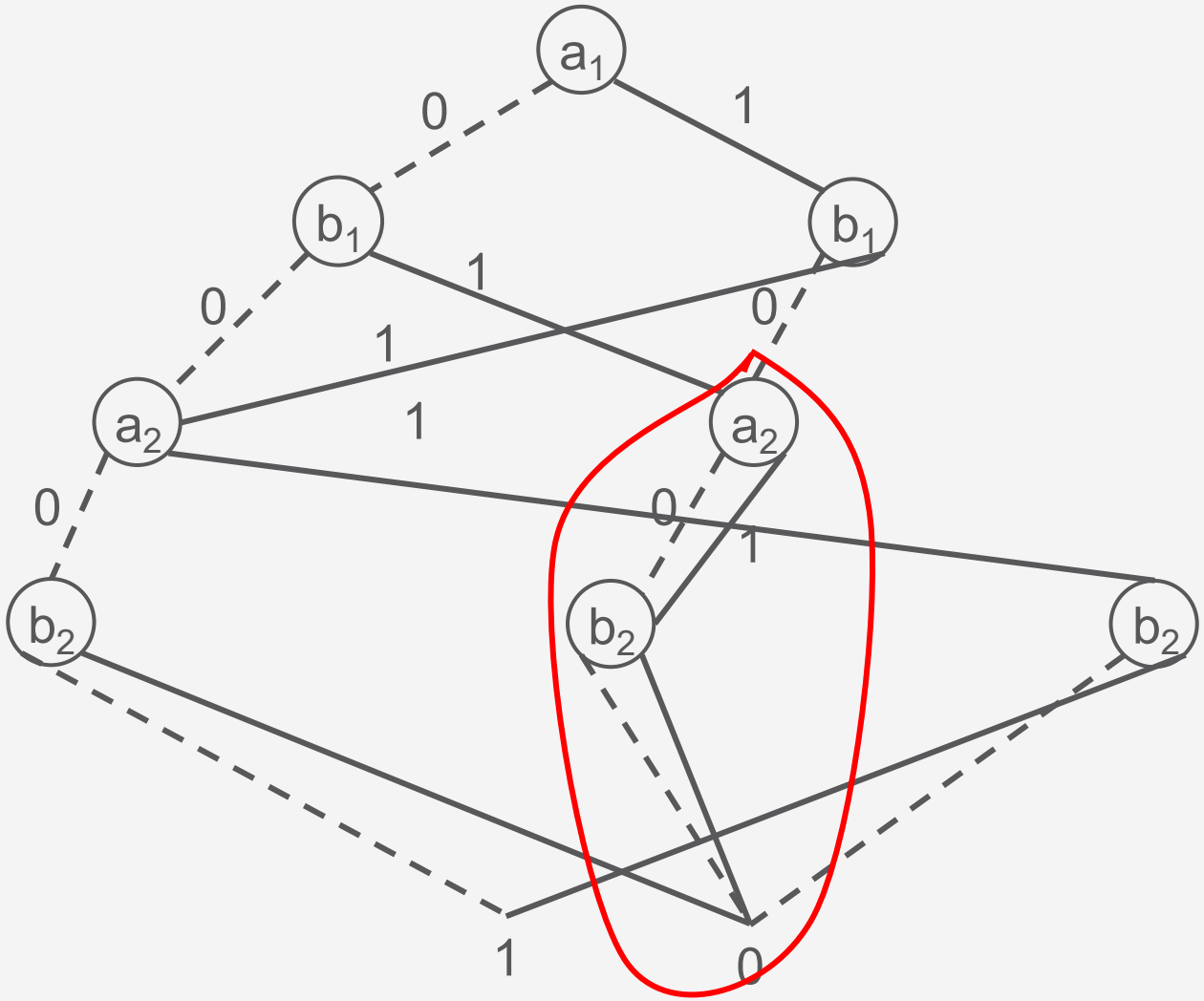
Entfernung redundanter nicht-terminaler Knoten



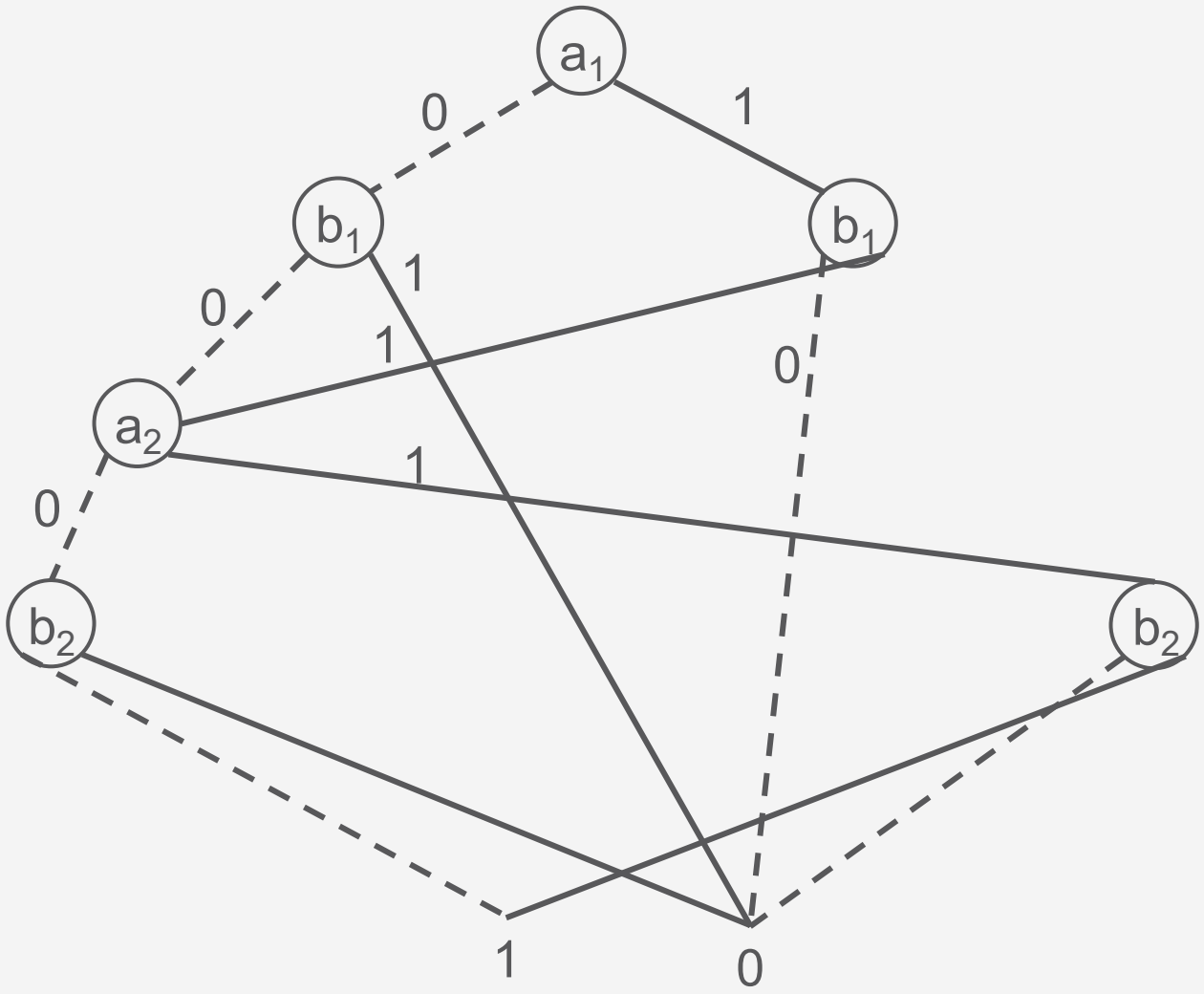
Entfernung redundanter nicht-terminaler Knoten



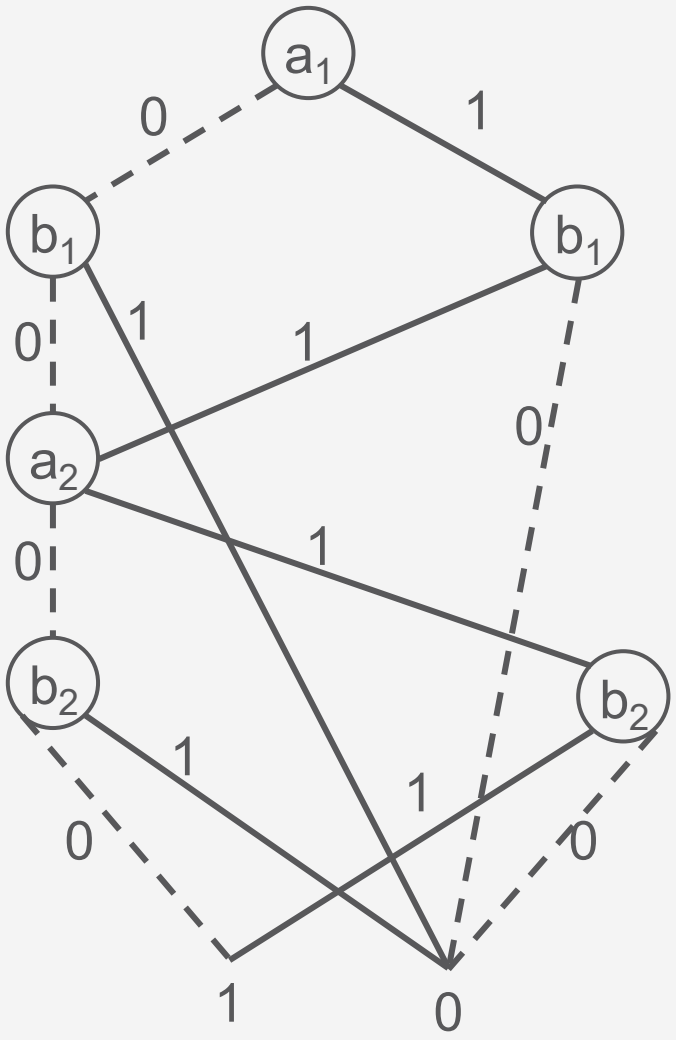
Entfernung redundanter Tests



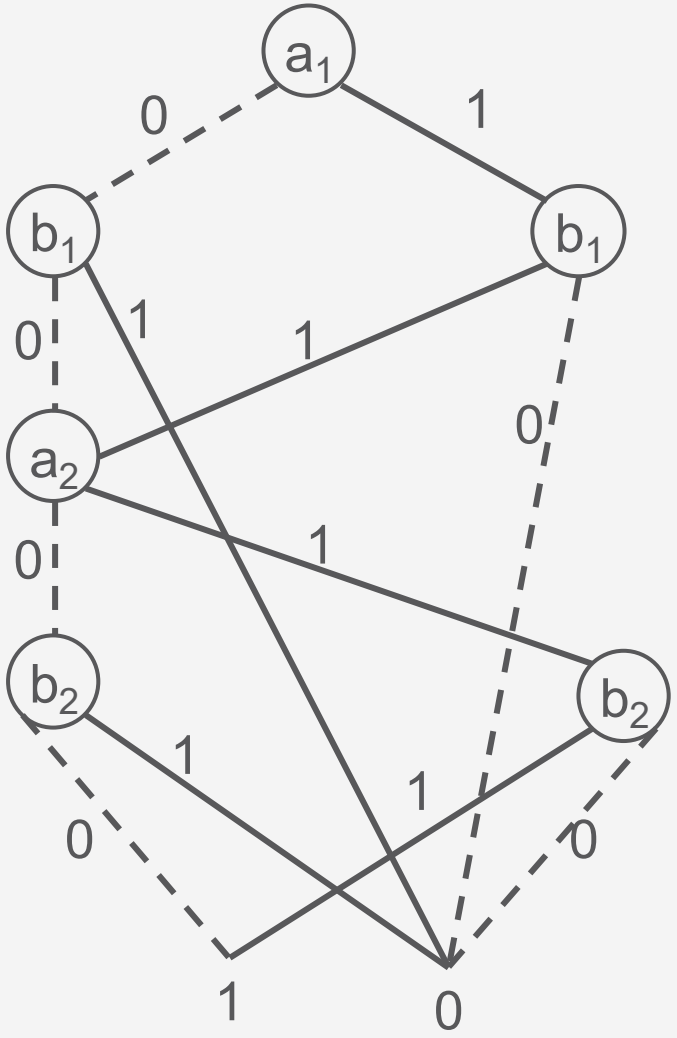
Entfernung redundanter Tests

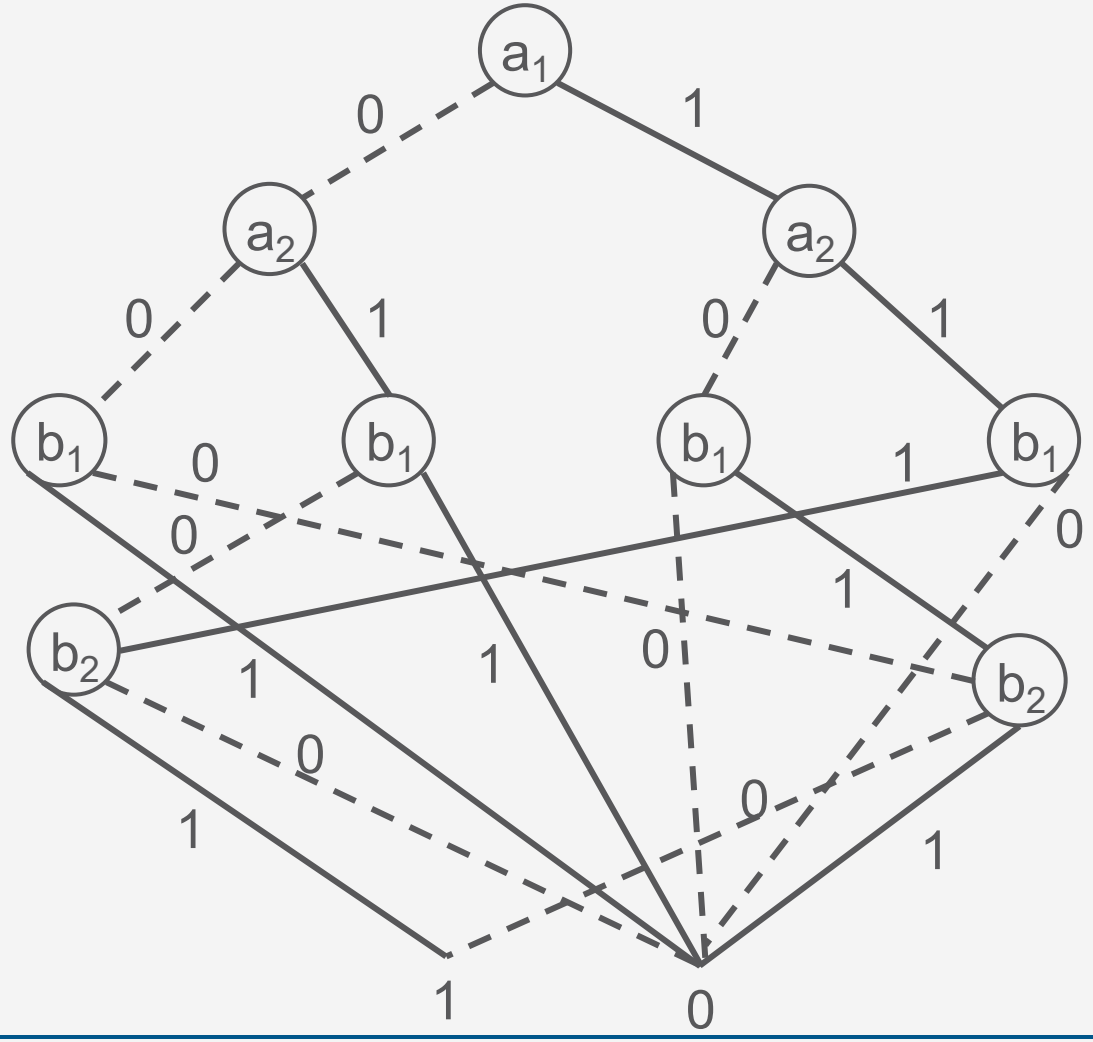


Ordered Binary Decision Diagram



Variablen-Reihenfolge in OBDDs





Variablen-Reihenfolge in OBDDs

- n-Bit Comparator:
  - $a_1 < b_1 < \ldots a_n < b_n \rightarrow 3 \cdot n + 2$ Knoten
  - $a_1 < \ldots < a_n < b_1 < \ldots < b_n \rightarrow 3 \cdot 2^n - 1$ Knoten
- eine optimale Reihenfolge zu finden ist ein NP-vollständiges Problem
- Heuristiken machen sich zum Beispiel die Struktur eines Schaltkreises zu Nutze ⇨ Abhängigkeitsanalyse

Eigenschaften von OBDDs

- kanonische Form
  - ⇨ Equivalence checking durch einfache Prüfung auf Isomorphie
  - ⇨ Satisfiability durch Vergleich mit einem OBDD, der nur eine 0 enthält
- effiziente Implementierung von Standard-Operationen wie Vereinigung, Schnittmenge, Vergleich …
- diese bewahren allerdings nicht die kanonische Form

Operationen auf OBDDs

- logische Operationen auf OBDDs können *rekursiv* definiert werden

⇨ Berechnungsaufwand ist linear in $|OBDD_1| \cdot |OBDD_2|$

- Zerlegung nach Shannon:

  $f = (\neg x \wedge f|_{x \leftarrow 0}) \vee (x \wedge f|_{x \leftarrow 1})$

⇨ Boolesche Funktionen können in zwei Teilprobleme zerlegt werden

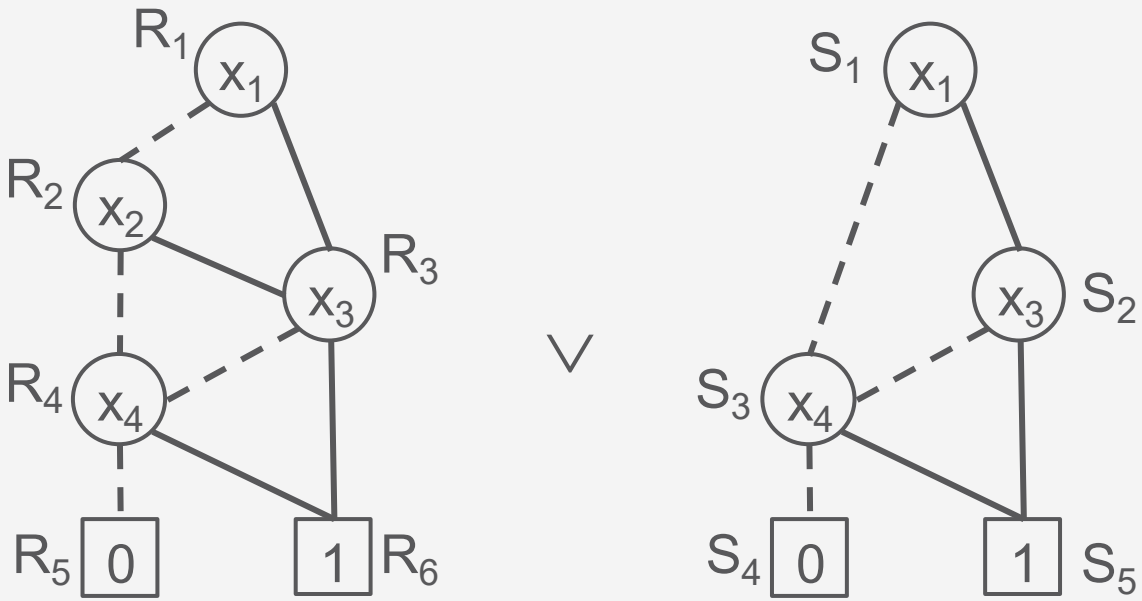
APPLY

Rekursiver Algorithmus zur Berechnung einer beliebigen binären logischen Operation $\otimes$ auf zwei OBDDs f und f' mit Wurzelknoten v and v'

1. Falls v und v' beides terminale Knoten sind, dann
   $f \otimes f' = value(v) \otimes value(v')$
2. Falls x = var(v) = var(v'), dann
   $f \otimes f' = (\neg x \wedge f|_{x \leftarrow 0} \otimes f'|_{x \leftarrow 0}) \vee (x \wedge f|_{x \leftarrow 1} \otimes f'|_{x \leftarrow 1})$
3. Falls x = var(v) und x < var(v') oder v' ein terminaler Knoten ist, dann
   $f \otimes f' = (\neg x \wedge f|_{x \leftarrow 0} \otimes f') \vee (x \wedge f|_{x \leftarrow 1} \otimes f')$
4. If x = var(v') and x < var(v) oder v ein terminaler Knoten ist, dan
   $f \otimes f' = (\neg x \wedge f \otimes f'|_{x \leftarrow 0}) \vee (x \wedge f \otimes f'|_{x \leftarrow 1})$

APPLY - Beispiel

1. Falls v und v' beides terminale Knoten sind, dann
   $f \otimes f' = value(v) \otimes value(v')$
2. Falls x = var(v) = var(v'), dann
   $f \otimes f' = (\neg x \wedge f|_{x \leftarrow 0} \otimes f'|_{x \leftarrow 0}) \vee (x \wedge f|_{x \leftarrow 1} \otimes f'|_{x \leftarrow 1})$
3. Falls x = var(v) und x < var(v') oder
   v' ein terminaler Knoten ist, dann
   $f \otimes f' = (\neg x \wedge f|_{x \leftarrow 0} \otimes f') \vee (x \wedge f|_{x \leftarrow 1} \otimes f')$
4. If x = var(v') and x < var(v) oder
   v ein terminaler Knoten ist, dan
   $f \otimes f' = (\neg x \wedge f \otimes f'|_{x \leftarrow 0}) \vee (x \wedge f \otimes f'|_{x \leftarrow 1})$

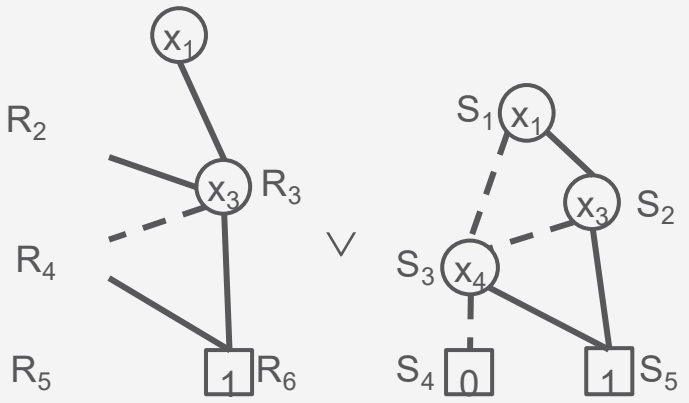


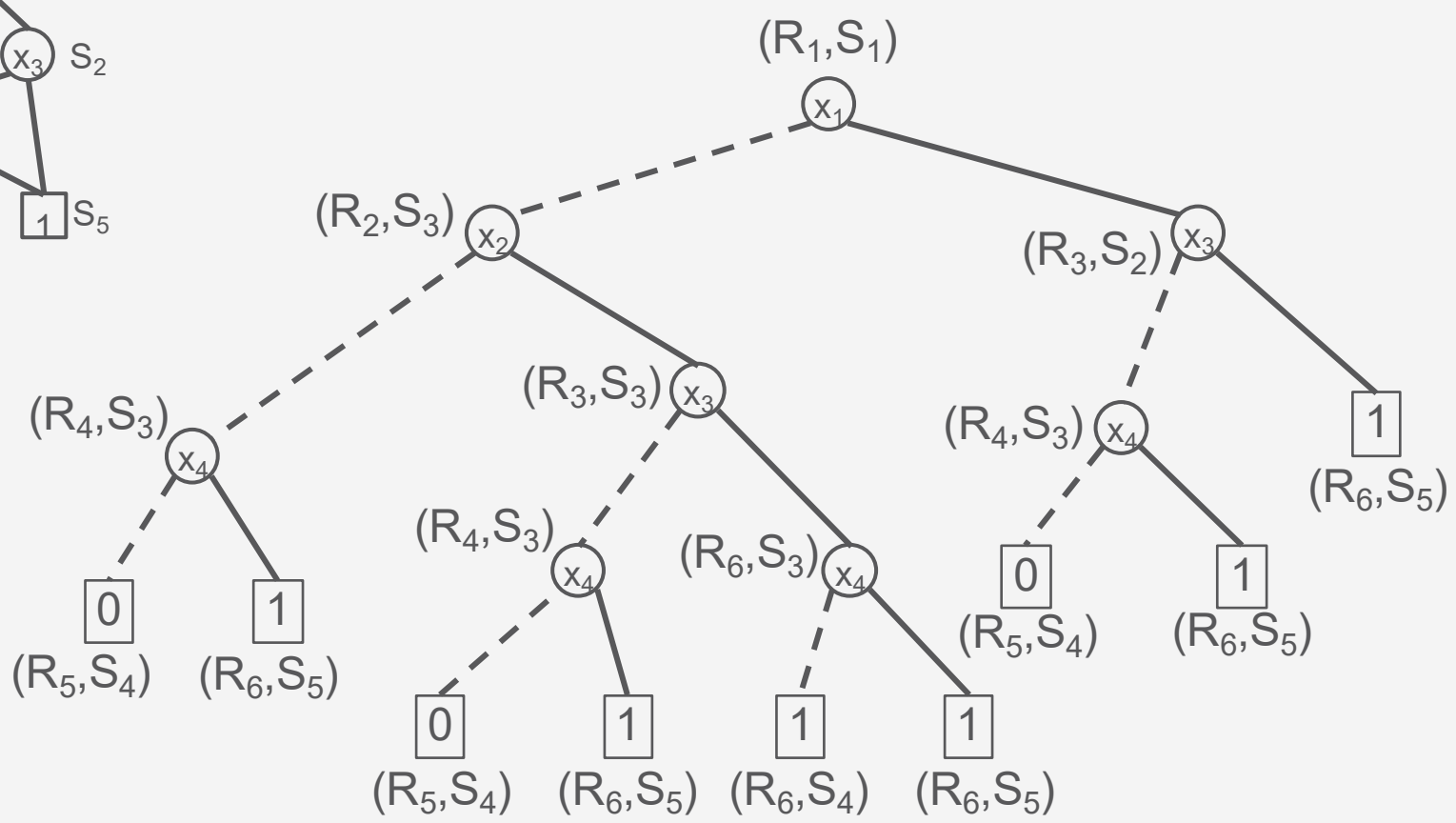
Wenden Sie den Algorithmus an!

APPLY - Beispiel

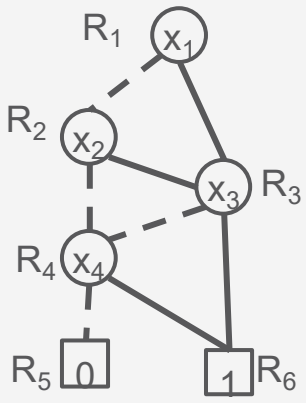
Shannon's expansion:

$$f \otimes f' = (\neg x \wedge f|_{x\leftarrow 0} \otimes f'|_{x\leftarrow 0}) \vee (x \wedge f|_{x\leftarrow 1} \otimes f'|_{x\leftarrow 1})$$

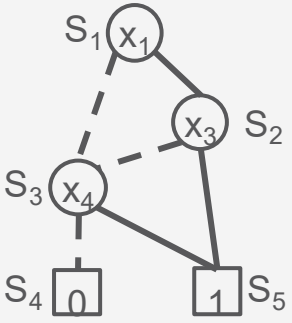


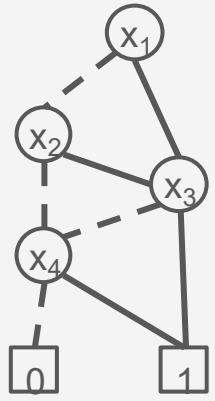


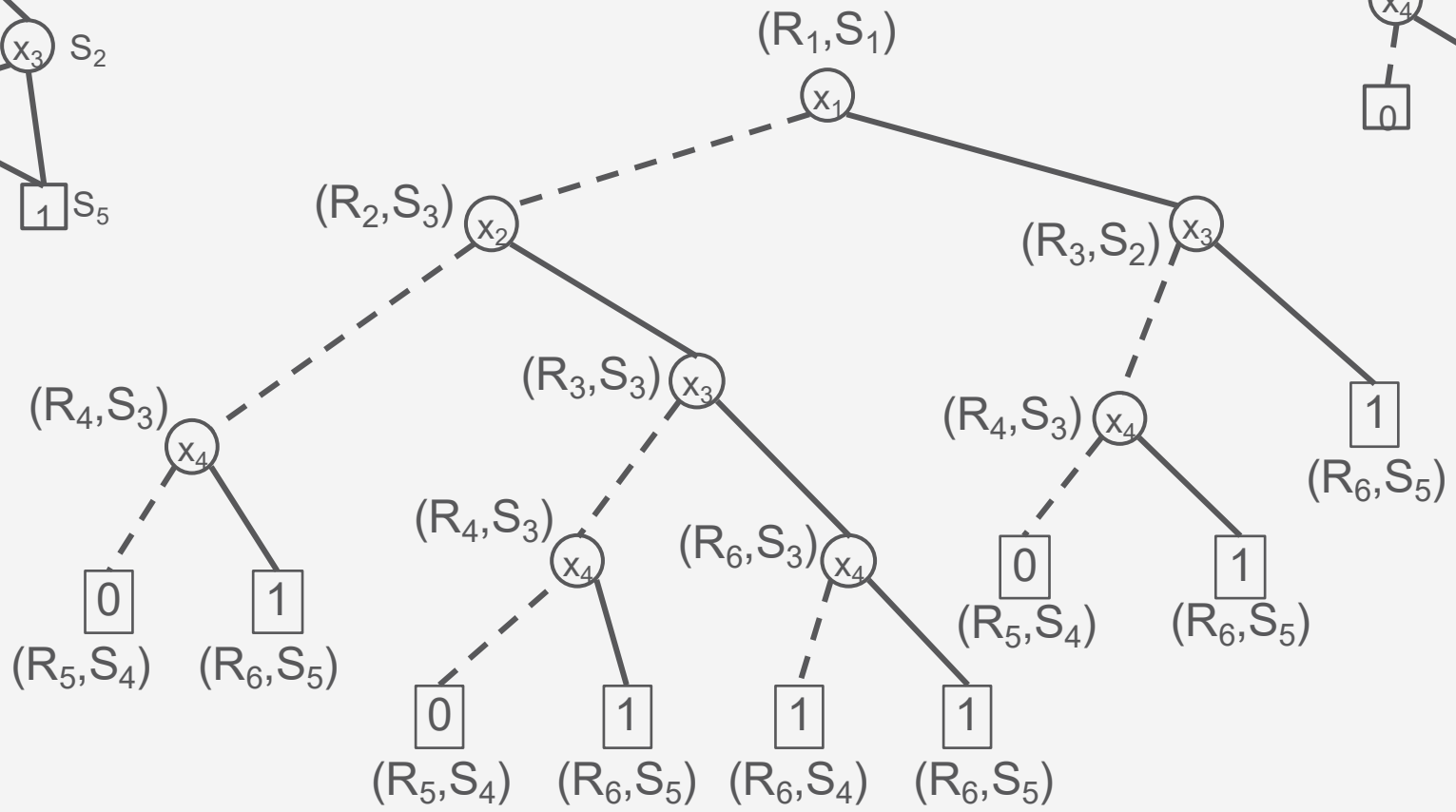
APPLY - Beispiel



$\vee$







Existentielle Quantifizierung

- Betrachte alle möglichen (Booleschen!) Werte von Variablen explizit

$\exists x : f = f|_{x \leftarrow 0} \vee f'|_{x \leftarrow 1}$

$\exists x, y\ [P(x, y)] \equiv \exists x [P(x, 0) \vee P(x, 1)] \equiv [\big(P(0,0) \vee P(0,1)\big) \vee \big(P(1,0) \vee P(1,1)\big)]$

Symbolisches Model Checking mit BDDs

- f = EX p
  mit Zustandsvariablen und Übergangsfunktion:

$$f(x_1, \dots, x_n) = \exists x_1', \dots x_n' \, [T(x_1, \dots, x_n, x_1', \dots, x_n') \wedge p(x_1', \dots, x_n')]$$

- Fixpunktberechnung: $EFp = \mu Z.\ p \vee EXZ$
  mit Zustandsvariablen und Übergangsfunktion:

$$EFp = \mu Z.\ p \vee \exists x_1', \dots x_n' \, [T(x_1, \dots, x_n, x_1', \dots, x_n')]$$

Überblick

- Einführung
- Vorüberlegungen: Eigenschaften von CTL
- Labeling-Algorithmus für CTL
- Model Checking als Fixpunktberechnung
- BDD-based Model Checking
- Zusammenfassung

Zusammenfassung

- Labeling Algorithmen zur Prüfung von CTL Formeln
  ⇨ schrittweise Markierung von Zuständen mit Teilformeln
- kompakte Formulierung als Fixpunktberechnung
- BDD-basiertes Model Checking
  - Kripke-Strukturen werden als Boolesche Formeln kodiert
  - Repräsentation der Booleschen Formeln als *Ordered Binary Decision Diagrams* (OBDD)
  - kompakte Repräsentation und effiziente Implementierung logischer Operationen
  - kanonische Form
    ⇨ einfache Prüfung von Äquivalenz und Erfüllbarkeit (*satisfiability*)
  - kann zur Fixpunktberechnung verwendet werden

Referenzen

- Edmund M. Clarke, Orna Grumberg und Doron A. Peled. ***Model Checking***
  MIT Press, 1999.
- Christel Baier und Joost-Pieter Katoen.
  Principles of Model Checking
  MIT Press, 2008.